MAKANAPP / BACKEND ENGINEERING

راهنمای جامع پروژه و بک‌اند ماکان

شناخت محصول، معماری، جریان اجرا و توسعه

نسخه بررسی: `main @ f0ff3be`

تاریخ بازبینی: `2026-10-12`

> کتاب فارسی برای توسعه‌دهنده بک‌اند؛ بر مبنای کد واقعی، SQL Server و قراردادهای فعلی. این فایل بدون اینترنت قابل مطالعه است. برای جست‌وجو از Ctrl+F استفاده کنید.

فهرست کتاب

- فهرست مطالب
- قابلیت‌ها و معماری
- از کجا شروع کنیم و پروژه چه مسئله‌ای حل می‌کند؟
  - نتیجه‌ای که خواننده باید بعد از مطالعه داشته باشد
  - برنامه مطالعه و اولین کار عملی
- واژه‌نامه مدل و شناسه‌ها
  - مثال چندسازمانی
- نقشه کد و مسئولیت لایه‌ها
  - از Controller به implementation
  - جای درست یک تغییر
- startup، Dependency Injection و مسیر هر درخواست
  - طول عمر سرویس‌ها
  - ترتیب pipeline
  - نشست این پروژه چگونه کار می‌کند؟
  - مجوز به صورت مرحله‌ای
  - خطا چگونه به HTTP تبدیل می‌شود؟
- راه‌اندازی محلی
  - پیش‌نیاز و بررسی اولیه
  - انتخاب دیتابیس توسعه
  - اعمال migration و اجرا
- تنظیمات
- قرارداد مشترک HTTP
  - استفاده در PowerShell و Postman
- ورود و پروفایل
  - دریافت OTP و نشست
  - تکمیل پروفایل و خروج
- فضای کاری و سرپرست
  - شناسه‌ها را با هم اشتباه نگیرید
  - انتخاب فضای شخصی یا سازمانی
  - دعوت و فرزند
- آموزش و حضور و غیاب
  - مجوزهای اصلی
  - ساخت کلاس
  - جلسه و برنامه تکرارشونده
  - ثبت و اصلاح حضور و غیاب
- تکلیف و ارزیابی
  - ساخت و انتشار
  - پاسخ و تحویل دانش‌آموز
  - ارزیابی و انتشار نمره
- آزمون و انتشار نمره
  - طراحی و نسخه‌بندی آزمون
  - شروع، ذخیره پاسخ و نهایی‌سازی
  - تصحیح و انتشار نمره آزمون
- فایل‌ها
- پیام‌رسانی و همگام‌سازی
  - آغاز گفت‌وگو و مجوز ارتباط
  - گروه، کانال و پیام
  - تاریخچه، Delta و رسید خواندن
  - SignalR
  - جست‌وجو، مسدودسازی و گزارش
- خطاها و تلاش مجدد
  - قراردادهای retry
- داشبوردهای نقش‌محور
- گزارش‌های تفصیلی نقش‌محور
- نقشه persistence و مرز تراکنش‌ها
  - سه لایه تضمین درستی داده
  - سه چیز متفاوت‌اند: revision، rowversion و operationId
  - نمونه‌های واقعی تراکنش و locking
  - migration و محیط توسعه
- شش مسیر واقعی برای مطالعه کد از ابتدا تا انتها
  - ۱. درخواست و تأیید OTP
  - ۲. آخرین ظرفیت کلاس
  - ۳. تحویل تکلیف و انتشار نمره
  - ۴. پاسخ آزمون تا نتیجه رسمی
  - ۵. ارسال پیام، Delta و SignalR
  - ۶. یک گزارش نقش‌محور
- روش توسعه و عیب‌یابی برای عضو جدید تیم
  - یک قابلیت کوچک را چطور اضافه یا اصلاح کنیم؟
  - تست‌ها چه چیزی را ثابت می‌کنند؟
  - مسیر عیب‌یابی یک درخواست ناموفق
  - مرز میزبانی و موارد لازم برای تحویل عملیاتی
- تست و عیب‌یابی
  - اجرای validation
  - عیب‌یابی سریع

- محدودیت‌ها و مسیر مطالعه
  - مرز قابلیت موجود
  - دیتابیس و منابع کد
- نگهداری همین راهنما و مرجع
- شناسنامه بازبینی و نتیجه اعتبارسنجی

راهنمای جامع پروژه و استفاده از بک‌اند و API ماکان

این راهنما برای تحویل پروژه به یک توسعه‌دهنده بک‌اند نوشته شده است: شناخت محصول و مدل، معماری و مسئولیت فایل‌ها، راه‌اندازی، استفاده از API، مطالعه مسیر اجرا، persistence، تست و ادامه توسعه. مبنای بازبینی `main@f0ff3be` در `2026-10-12` است: ۱۴۱ عملیات HTTP در ۲۲ Controller، شامل داشبوردها، گزارش‌های تفصیلی OTP، STEP 11B مدیریت‌شده و میزبانی Production.

برای شروع، بخش راه‌اندازی و ورود را بخوانید؛ سپس فضای کاری مناسب را انتخاب کنید و جریان موردنیاز را دنبال کنید. مرجع کامل Endpointها و قراردادها همراه این راهنماست و مسیر، پارامتر، بدنه، نوع پاسخ و فیلدهای تمام APIهای Controllerها را نشان می‌دهد.

فهرست مطالب

- قابلیت‌ها و معماری
- شروع مطالعه و هدف پروژه
- واژه‌نامه مدل و شناسه‌ها
- نقشه کد و مسئولیت لایه‌ها
- DI، startup و pipeline
- راه‌اندازی محلی
- تنظیمات
- قرارداد مشترک HTTP
- ورود و پروفایل
- فضای کاری و سرپرست
- آموزش و حضور و غیاب
- تکلیف و ارزیابی
- آزمون و انتشار نمره
- فایل‌ها
- پیام‌رسانی و همگام‌سازی
- خطاها و تلاش مجدد
- داشبوردهای نقش‌محور
- گزارش‌های تفصیلی
- استقرار و secretهای Production
- قرارداد اتصال React و React Native

- persistence و تراکنش
- شش مسیر واقعی مطالعه کد
- روش توسعه و عیب‌یابی
- تست و عیب‌یابی
- نگهداری و ساخت HTML/PDF
- نتیجه اعتبارسنجی
- محدودیت‌ها و مسیر مطالعه

قابلیت‌ها و معماری

بخش	قابلیت موجود
Identity	درخواست و تأیید OTP، نشست Bearer، خروج، مشاهده و تکمیل پروفایل
Organization	فهرست و انتخاب فضای کاری، مشاهده و پذیرش یا رد دعوت
Guardian	فهرست فرزندان مجاز، انتخاب فرزند و مشاهده رابطه
Academic	دوره، درس، کلاس، ثبت‌نام، انتساب معلم، برنامه تکرارشونده، جلسه و حضور و غیاب
Assignment	پیش‌نویس و انتشار تکلیف، پاسخ متنی و فایل، تحویل نهایی، ارزیابی و انتشار یا اصلاح نمره
Exam	نسخه‌بندی آزمون، سؤال تستی و تشریحی، شروع تلاش، ذخیره پاسخ، انتقال حق نوشتن، نهایی‌سازی، تصحیح و انتشار نمره
Storage	بارگذاری، metadata، دانلود مجاز و حذف فایل
Messaging	گفت‌وگوی مستقیم، گروه، کانال، مدیریت اعضا، پیام و رسانه، جست‌وجو، Delta، Report، Block و SignalR
Reports	چهار داشبورد مخاطب و شانزده گزارش تفصیلی؛ داده مجاز نقش، pagination، حضور و نمره منتشرشده

پروژه یک Modular Monolith با Onion Architecture است. همه قابلیت‌ها در یک میزبان API اجرا می‌شوند و SQL Server مرجع داده‌های تراکنشی است.

پروژه	مسئولیت
`MakanApp.Domain`	مدل و قانون دامنه، بدون EF Core یا HTTP
`MakanApp.Application`	سرویس کاربردی، قرارداد ورودی و خروجی، اعتبارسنجی و مجوز منبع

پروژه	مسئولیت
`MakanApp.Infrastructure`	پیاده‌سازی portها، SQL Server، EF Core 9، فایل و adapterهای فنی
`MakanApp.Api`	Controller، احراز هویت، SignalR، ProblemDetails و ترکیب DI
`tests`	تست‌های Unit، Integration و Architecture

```
HTTP -> Controller -> Application Service -> Domain rules / Application port
                                          -> Infrastructure -> SQL Server
```

وابستگی کامپایل از بیرون به داخل است: `Api -> Application`، `Api -> Infrastructure`، `Infrastructure ->` `Application/Domain` و `Application -> Domain`. مسیر اجرای درخواست با جهت وابستگی یکسان نیست؛ Application از interface استفاده می‌کند و DI پیاده‌سازی Infrastructure را به آن می‌دهد.

از کجا شروع کنیم و پروژه چه مسئله‌ای حل می‌کند؟

MakanApp یک backend مشترک برای ارتباط و فعالیت آموزشی در سازمان است. یک شخص می‌تواند حساب ورود داشته باشد، عضو چند سازمان باشد، در هر سازمان چند نقش داشته باشد و در نشست جاری فقط یک فضای کاری و نقش را انتخاب کند. مدرسه/آموزشگاه، کلاس، حضور، تکلیف و آزمون داده رسمی آموزشی‌اند؛ پیام‌رسان مسیر ارتباط افراد است. داشبورد و گزارش، وضعیت همین داده رسمی را برای مخاطب مجاز خلاصه می‌کنند.

در این نسخه ناحیه School و Messenger جریان‌های واقعی دارند. نام Commerce، Intelligence، Knowledge، Copilot، Notification و Platform Operations در معماری هدف آمده است، اما وجود آن نام‌ها به معنی وجود API اجرایی آن‌ها نیست. مدیر مدرسه `Manager` با ادمین پلتفرم یک مفهوم نیست؛ endpoint عمومی مدیریت پلتفرم یا provisioning سازمان در این مخزن وجود ندارد.

نتیجه‌ای که خواننده باید بعد از مطالعه داشته باشد

1. بتواند solution را بسازد، دیتابیس توسعه را آگاهانه انتخاب کند و API را اجرا کند.
2. فرق حساب ورود، شخص، پرونده سازمانی، عضویت، نقش، ثبت‌نام و رابطه والد را بداند.
3. از یک endpoint به Controller، Application service، port، implementation و configuration برسد.
4. بداند کدام تغییر به Infrastructure، Application، Domain یا Api تعلق دارد.
5. خطای ورودی، نداشتن مجوز، تضاد وضعیت، تضاد نسخه و خرابی provider را تفکیک کند.
6. یک تغییر کوچک را با تست مناسب، migration بررسی‌شده و قرارداد HTTP روشن تحویل دهد.

برنامه مطالعه و اولین کار عملی

ترتیب	مطالعه/عمل	نشانه فهم درست
۱	همین بخش، معماری و واژه‌نامه	می‌توانید مدل یک Student و Parent در دو سازمان را توضیح دهید
۲	راه‌اندازی و تنظیمات	host بالا می‌آید و OpenAPI از همان build خوانده می‌شود
۳	ورود، workspace و child context	context از نشست معتبر سرور می‌آید؛ role ورودی مجوز مستقل نیست
۴	یک جریان Academic یا Assignment	از HTTP تا ذخیره و برگشت DTO را در کد دنبال می‌کنید
۵	تراکنش، نسخه و idempotency	می‌دانید دو درخواست همزمان یا retry چه اثری دارند
۶	Messaging و outbox، سپس آزمون	فرق finalize، draft، notification، commit و release روشن است
۷	تست‌ها، عیب‌یابی، راهنمای توسعه	می‌توانید تستی بنویسید که رفتار واقعی مورد تغییر را ثابت کند

اجرای نخست را با `/health`، OpenAPI و تست‌ها شروع کنید. پایگاه تازه بعد از migration سازمان، عضویت یا کلاس آماده ندارد. ورود OTP یک حساب ایجاد می‌کند، اما نقش مدیر سازمان به آن نمی‌دهد. برای یادگیری جریان کامل سازمانی از fixture و تست‌ها استفاده کنید؛ داده fixture را با seed عمومی محصول اشتباه نگیرید.

واژه‌نامه مدل و شناسه‌ها

اصطلاح واقعی	معنی و محل	اشتباه رایج
`User`	حساب سراسری ورود در Identity	یک نقش سازمانی ثابت روی User وجود ندارد
`UserCredential`	اتصال روش ورود، فعلاً موبایل، به User	شماره موبایل شناسه پرونده دانش‌آموز نیست
`Person`	مشخصات شخص مانند نام؛ مستقل از سازمان	ممکن است پرونده آموزشی وجود داشته باشد ولی حساب ورود متصل هنوز نداشته باشد
`OrganizationPerson`	پرونده همان Person در یک Organization	شناسه آن با PersonId و UserId قابل جایگزینی نیست
`Membership`	رابطه User با سازمان	پرونده شخص و اجازه استفاده حساب دو رابطه متفاوت‌اند

اصطلاح واقعی	معنی و محل	اشتباه رایج
`RoleAssignment`	نقش معتبر یک Membership؛ مثل Student/Teacher/Parent/Manager	داشتن نقش در سازمان A مجوز سازمان B نیست
`UserSession`	نشست دارای hash توکن، زمان انقضا و workspace/child منتخب	Bearer این پروژه JWT خودبسنده نیست
`AccessContext`	نتیجه معتبر resolve نشست، سازمان، عضویت، نقش و subject	DTO context یا مقدار ذخیره‌شده در کلاینت اثبات مجوز درخواست بعدی نیست
`GuardianRelation`	رابطه زمان‌دار والد/سرپرست با پرونده فراگیر در سازمان	Parent بودن به‌تنهایی مجوز همه دانش‌آموزان نیست
`Course` / `AcademicPeriod`	دوره زمانی آموزشی / درس سازمان	هیچ‌کدام جای Class را نمی‌گیرند
`AcademicClass`	کلاس با درس، دوره، ظرفیت و وضعیت	enrollment به کلاس مشخص تعلق دارد
`Enrollment`	ثبت‌نام پرونده فراگیر در کلاس	UserId را به جای LearnerOrganizationPersonId نفرستید
`TeacherAssignment`	انتساب Membership معلم به کلاس	role Teacher به‌تنهایی دسترسی به همه کلاس‌ها نمی‌دهد
`Session`	جلسه آموزشی با StartUtc/EndUtc	با UserSession ورود اشتباه نشود
`ScheduleRule`	الگوی تکرار با ساعت محلی و TimeZoneId	زمان محلی الگو با زمان UTC رخداد یکی نیست
`AssignmentVersion` / `AssignmentRecipient`	نسخه تکلیف و گیرندگان تثبیت‌شده انتشار	تغییر enrollment بعدی، گیرنده تاریخی را خودکار بازنویسی نمی‌کند
`SubmissionAttempt`	پاسخ/تلاش تحویل تکلیف	جدول تلاش آزمون نیست
`GradeRelease` / `EvaluationRevision`	نسخه ارزیابی تکلیف / انتشار رسمی آن	draft ارزیابی نتیجه عمومی دانش‌آموز نیست
`QuestionVersion` / `ExamVersion`	تعریف نسخه‌دار آزمون و سؤال	نسخه منتشرشده را برای اصلاح پاسخ یک دانش‌آموز تغییر ندهید
`ExamAttemptQuestion` / `ExamAttempt`	تلاش دانش‌آموز و مجموعه/ترتیب سؤال تثبیت‌شده	تلاش با deadline و شمارنده نسخه خود کار می‌کند
`ExamFinalAnswer` / `AnswerRevision`	تاریخچه پاسخ پذیرفته‌شده / snapshot پاسخ نهایی	autosave با finalize یک عملیات نیست
`ExamGradeRelease` / `ExamGradeRevision`	نسخه تصحیح آزمون / انتشار رسمی آن	با جدول نمره تکلیف یکسان نیست
`FileAsset`	metadata و scope فایل؛ bytes در IFileStorage	FileId بدون مجوز منبع اجازه دانلود نمی‌دهد
`ConversationParticipant`	عضویت و نقش ارتباطی در گفتگو	Owner/Admin/Member گفتگو با نقش آموزشی سازمان متفاوت است

اصطلاح واقعی	معنی و محل	اشتباه رایج
`MessagingChangeEvent` / `RealtimeOutbox`	تغییر پایدار برای Delta / کار ارسال notification	دریافت SignalR شرط ثبت پیام نیست

مثال چندسازمانی

یک User می‌تواند در سازمان A نقش Teacher و در سازمان B نقش Parent داشته باشد. انتخاب workspace A/Teacher در نشست فقط عملیات مجاز Teacher در A را فعال می‌کند. برای مشاهده فرزند در B باید workspace B/Parent و child معتبر آن انتخاب شود. حتی اگر ClassId معتبر A در درخواست B ارسال شود، سرویس باید scope جاری و رابطه منبع را کنترل کند. FK مرکب از اتصال داده دو سازمان جلوگیری می‌کند؛ مجوز خواندن HTTP همچنان وظیفه سرویس است.

نقشه کد و مسئولیت لایه‌ها

پروژه‌های اصلی در ریشه solution هستند؛ پوشه `src` وجود ندارد. namespaceها و نام کلاس‌ها انگلیسی‌اند؛ توضیحات توسعه‌دهنده فارسی است. هر حوزه پوشه‌های متناظر خودش را دارد، اما میزبان و دیتابیس مشترک‌اند.

```
MakanApp.Api/Controllers          HTTP transport
MakanApp.Application/{Area}       Use cases + DTOs + ports + policies
MakanApp.Domain/{Area}            Entities + enums + rules
MakanApp.Infrastructure/{Area}    Adapters + EF stores
MakanApp.Infrastructure/Persistence/Configurations/{Area}
MakanApp.Infrastructure/Persistence/Migrations
tests/MakanApp.UnitTests
tests/MakanApp.IntegrationTests
tests/MakanApp.ArchitectureTests
```

از Controller به implementation

حوزه/عملیات	Application service	port و implementation اصلی
OTP، پروفایل، نشست	`IdentityService`	`IIdentityStore → EfIdentityStore`؛ `IIdentitySecurity → IdentityCryptography`؛ `IOtpDeliveryProvider`
workspace و دعوت	`OrganizationService`	`IOrganizationStore → EfOrganizationStore`
والد و انتخاب فرزند	`GuardianService`	`IGuardianStore → EfGuardianStore`
context مشترک	`GuardianAccessContextResolver`	`IAccessContextResolver`؛ ابتدا OrganizationService، سپس اعتبار child

حوزه/عملیات	Application service	port و implementation اصلی
دوره، درس، کلاس و ثبت‌نام	`AcademicService`	`IAcademicStore → EfAcademicStore`
جلسات، الگو و حضور	`AcademicSessionService`	`IAcademicSessionStore → EfAcademicSessionStore`
تألیف و انتشار تکلیف	`AssignmentService`	`IAssignmentStore → EfAssignmentStore`
پاسخ و تحویل تکلیف	`SubmissionService`	`ISubmissionStore → EfSubmissionStore`
تصحیح تکلیف	`EvaluationService`	`IEvaluationStore → EfEvaluationStore`
تألیف آزمون	`ExamService`	`IExamStore → EfExamStore`
شروع تلاش آزمون	`ExamAttemptService`	`IExamAttemptStore → EfExamAttemptStore`
lease و پاسخ آزمون	`ExamAnswerService`	`IExamAnswerStore → EfExamAnswerStore`
نهایی‌سازی آزمون	`ExamFinalizationService`	`IExamFinalizationStore → EfExamFinalizationStore`
تصحیح آزمون	`ExamGradingService`	`IExamGradingStore → EfExamGradingStore`
گفتگو و پیام	`MessagingService`	`IMessagingStore → EfMessagingStore`
گروه، کانال و مدیریت اعضا	`ConversationManagementService`	`IConversationManagementStore → EfMessagingStore`
Block و Report	`MessagingSafetyService`	`IMessagingSafetyStore → EfMessagingStore`
فایل	`StorageService`	`IFileAssetStore → EfFileAssetStore`؛ `IFileStorage`؛ `IFileAssetBoundAccessResolver`
داشبورد	`DashboardService`	`IDashboardReadService → EfDashboardReadService`
گزارش تفصیلی	`DetailedReportService`	`IDetailedReportReadService → EfDetailedReportReadService`

فایل‌های `*Abstractions.cs` قرارداد میان use case و ابزار فنی را نگه می‌دارند. فایل‌های `*Contracts.cs` شکل ورودی/خروجی‌اند؛ entity EF را مستقیم به کلاینت برنمی‌گردانیم. بعضی کلاس‌ها `partial` هستند: هنگام مطالعه یک service یا store همه فایل‌های هم‌نام آن را بررسی کنید؛ پیاده‌سازی ممکن است در فایل اصلی تمام نشود.

جای درست یک تغییر

تغییر موردنیاز	محل معمول	دلیل
قانون وضعیت، اعتبار ذاتی entity، محاسبه خالص	Domain	مستقل از HTTP و persistence قابل استفاده و تست باشد
ترتیب use case، مجوز منبع، DTO و اعتبارسنجی ترکیبی	Application	عملیات کسب‌وکاری و portهای لازم را تعیین می‌کند
SQL، EF mapping، locking، فایل و provider	Infrastructure	وابستگی ابزار فنی از لایه داخلی جدا می‌ماند
route، binding، status، header، authentication handler	Api	قرارداد انتقال و composition root است
schema جدید	Configuration + Migration	تغییر باید نسخه‌بندی‌شده و قابل بازتولید باشد

در این مخزن Event Sourcing، MediatR، GenericRepository یا سرویس‌های مستقل برای هر حوزه استفاده نشده‌اند. برای افزودن رفتار، ابتدا الگوی همان service و port موجود را دنبال کنید. interface تازه زمانی لازم است که یک وابستگی واقعی را جدا کند.

startup، Dependency Injection و مسیر هر درخواست

دو نقطه اصلی ترکیب سیستم Program.cs و DependencyInjection.cs هستند. Program سرویس‌های HTTP را اضافه می‌کند؛ AddInfrastructure اتصال port، SQLها، serviceها، options و adapterها را متصل می‌کند. Controller فقط interface سرویس را دریافت می‌کند؛ Controller از EF query استفاده نمی‌کند.

طول عمر سرویس‌ها

- `MakanDbContext` و بیشتر service/storeها Scoped هستند: در scope درخواست از context مشترک استفاده می‌کنند؛ context بین درخواست‌ها share نمی‌شود.
- `TimeProvider.System`، policy، optionsهای بدون state درخواست و بعضی adapterها Singleton هستند. user/context درخواست را داخل singleton ذخیره نکنید.
- `MeliPayamakOtpProvider` از HttpClient ثبت‌شده و lifetime فنی خودش استفاده می‌کند؛ HttpClient را برای هر درخواست دستی نسازید.
- `EfMessagingStore` یک‌بار در scope ثبت و برای سه interface resolve می‌شود. سه interface به معنی سه DbContext مستقل نیست.
- dispatcher پس‌زمینه Singleton میزبان است ولی برای هر دور `CreateAsyncScope` می‌سازد تا processor و DbContext Scoped را درست دریافت کند.

ترتیب pipeline

```
Request
-> Exception handling / technical endpoints
-> HTTPS redirection (outside Testing)
-> Routing
-> CORS
-> Rate limiter
-> Session authentication
-> ASP.NET authorization
-> Controller -> Application -> Domain/port -> Infrastructure
-> DTO / HTTP result
```

`[Authorize]` فقط الزام ورود را در transport اعمال می‌کند. کنترل نقش، سازمان و رابطه کلاس/فرزند داخل Application ادامه دارد. CORS قبل از authentication است تا preflight مرورگر پردازش شود. محدودیت OTP یک bucket حافظه‌ای بر IP است؛ محدودیت شماره موبایل از persisted challengeها نیز بررسی می‌شود. این دو محدودیت یک شمارنده نیستند.

نشست این پروژه چگونه کار می‌کند؟

`SessionAuthenticationHandler` مقدار header `Authorization: Bearer ...` را به `IdentityService.AuthenticateAsync` می‌دهد. `IdentityCryptography` از توکن تصادفی ۳۲ بایتی، hash HMAC-SHA256 می‌سازد؛ فقط hash در `UserSessions` نگه‌داری می‌شود. احراز هویت، نشست فعال/منقضی/لغوشده و User مربوط را از سمت سرور بررسی می‌کند. claims شامل UserId و SessionId هستند؛ role سازمانی از claims ثابت استخراج نمی‌شود.

توکن جاری JWT نیست؛ decode کردن آن با ابزار JWT یا نگه‌داری role کلاینت مبنای تصمیم سرور نیست. refresh token جدا وجود ندارد. خروج نشست را revoke می‌کند؛ تغییر `Identity:SecurityKey` هم hash OTPها و هم نشست‌های قبلی را نامعتبر می‌کند.

مجوز به صورت مرحله‌ای

```
Bearer -> active UserSession -> UserId + SessionId
-> OrganizationService: workspace + organization + membership + active role
-> GuardianAccessContextResolver: selected child relation if present
-> use case: resource organization + ownership/enrollment/teacher assignment
-> authorized projection or mutation
```

`GuardianAccessContextResolver` در Application قرار دارد؛ به پیاده‌سازی SQL وابسته نیست. انتخاب workspace در `OrganizationService`، child قبلی نشست را پاک می‌کند. رابطه منتخب در درخواست بعدی دوباره بررسی می‌شود؛ revoke شدن رابطه نباید با cache قبلی دور زده شود.

خطا چگونه به HTTP تبدیل می‌شود؟

`GlobalExceptionHandler` exceptionهای حوزه را به ProblemDetails با `code` پایدار و status تبدیل می‌کند. خطای ناشناخته با `UNEXPECTED_ERROR` و پیام عمومی500 برمی‌گردد؛ stack trace پاسخ عمومی نیست. `traceId` به پاسخ اضافه می‌شود تا بتوان درخواست را در log پیدا کرد. خطای model binding قبل از رسیدن به use case ممکن است ValidationProblemDetails دارای `errors` بدهد؛ وجود `code` اختصاصی برای همه خطاهای binding تضمین نشده است.

`CancellationToken` از RequestAborted/پارامتر Controller تا I/O عبور می‌کند. لغو درخواست، retry موفق یا rollback همه side effectهای بیرونی را به‌تنهایی تضمین نمی‌کند؛ ارسال SMS و نوشتن فایل باید با قرارداد هر عملیات بررسی شوند.

راه‌اندازی محلی

پیش‌نیاز و بررسی اولیه

- .NET SDK سازگار با هدف `net9.0`؛ baseline پروژه .NET 9 است.
- SQL Server محلی یا یک پایگاه توسعه مجاز؛ تست‌های Integration مشخصاً به Windows و SQL Server LocalDB نیاز دارند.
- دسترسی به NuGet برای restore اولیه.
- برای کار با migration، ابزار محلی `dotnet-ef` نسخه `9.0.5` در `.config/dotnet-tools.json` تعریف شده است.

از ریشه مخزن اجرا کنید:

```
dotnet --list-sdks
dotnet restore MakanApp.sln
dotnet tool restore
dotnet build MakanApp.sln --configuration Debug --nologo
```

انتخاب دیتابیس توسعه

کلید اتصال `ConnectionStrings:MakanDatabase` است. نمونه زیر فقط از Windows Authentication و یک نام مشخص برای دیتابیس توسعه استفاده می‌کند:

```
$env:ConnectionStrings__MakanDatabase =
'Server=.;Database=MakanApp_Development;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertifi
cate=True'
```

این متغیر فقط برای PowerShell فعلی و فرایندهای فرزند آن تنظیم می‌شود. برای ذخیره تنظیم runtime در توسعه می‌توان از User Secrets استفاده کرد:

```
dotnet user-secrets set 'ConnectionStrings:MakanDatabase' $env:ConnectionStrings__MakanDatabase --project
```

```
MakanApp.Api/MakanApp.Api.csproj
```

`MakanDbContextFactory` هنگام اجرای ابزار EF، متغیر محیطی بالا را می‌خواند و User Secrets پروژه API را نمی‌خواند. در نبود متغیر، به `Server=.;Database=MakanApp` برمی‌گردد. برای اینکه migration روی مقصد دیگری اعمال نشود، متغیر اتصال را در همان پنجره‌ای که فرمان EF اجرا می‌شود صریح تنظیم کنید.

اعمال migration و اجرا

پس از انتخاب دیتابیس توسعه مجاز، migrationها را با فرمان زیر اعمال کنید. این فرمان schema مقصد را تغییر می‌دهد؛ مقصد آن باید همان پایگاه توسعه انتخاب‌شده باشد.

```
dotnet tool run dotnet-ef database update --project MakanApp.Infrastructure/MakanApp.Infrastructure.csproj --
startup-project MakanApp.Infrastructure/MakanApp.Infrastructure.csproj
dotnet dev-certs https --trust
dotnet run --project MakanApp.Api/MakanApp.Api.csproj --configuration Debug --launch-profile https
```

برنامه migration را خودکار در startup اجرا نمی‌کند. آخرین migration موجود هنگام نگارش، `20261007075650_AddExamGradingAndRelease` است. اعمال migration به معنی ایجاد سازمان، دعوت، دانش‌آموز یا حساب مدیر نیست؛ seed عمومی و Endpoint راه‌اندازی سازمان در پروژه وجود ندارد.

نشانی محلی	کاربرد
`https://localhost:7063/health`	زنده‌بودن میزبان؛ بررسی اتصال SQL Server در آن ثبت نشده است
`https://localhost:7063/swagger`	Swagger UI؛ مسیر نسبی در تمام محیط‌ها از جمله Production عمومی است
`https://localhost:7063/openapi/v1.json`	سند OpenAPI؛ مسیر نسبی در تمام محیط‌ها از جمله Production عمومی است
`https://localhost:7063/api/v1`	پیشوند APIهای کاربردی
`https://localhost:7063/hubs/messaging`	SignalR پیام‌رسانی

پروفایل HTTP ،HTTPS را هم روی `http://localhost:5077` باز می‌کند. خارج از محیط `Testing`، HTTPS redirection فعال است؛ برای نمونه‌های این سند از HTTPS استفاده کنید.

تنظیمات

مقادیر زیر از `appsettings.json` و `DependencyInjection.cs` آمده‌اند. برای متغیر محیطی، `:` را با `__` جایگزین کنید؛ مثلاً `Storage__MaxFileSizeBytes`.

کلید	پیش‌فرض یا رفتار
`ConnectionStrings:MakanDatabase`	در تنظیم پایه خالی است؛ Development نمونه اتصال محلی دارد
`Identity:SecurityKey`	حداقل ۳۲ بایت UTF-8؛ خارج از Development/Testing و هنگام انتخاب `MeliPayamakOtp` در هر محیط الزامی است؛ فقط provider محلی توسعه می‌تواند کلید تصادفی process داشته باشد
`Identity:Otp:LifetimeSeconds`	`300` ، اعتبار پنج‌دقیقه‌ای OTP
`Identity:Otp:ResendDelaySeconds`	`30` ، فاصله ارسال مجدد؛ در Testing صفر
`Identity:Otp:RateLimitWindowSeconds`	`3600`
`Identity:Otp:MaxRequestsPerWindow`	`5` درخواست برای شماره در پنجره تعیین‌شده در تنظیم پایه
`Identity:Otp:MaxFailedAttempts`	`5` تلاش ناموفق
`Identity:Otp:MinimumCodeLength` / `MaximumCodeLength`	`4` / `64` ؛ کد provider به‌صورت string opaque اعتبارسنجی می‌شود
`Sms:Provider`	`Development` در توسعه، `MeliPayamakOtp` برای سرویس واقعی، یا `Unavailable`
`Sms:MeliPayamakOtp:BaseUrl`	`https://console.melipayamak.com`
`Sms:MeliPayamakOtp:ServiceToken`	secret الزامی برای provider واقعی؛ هرگز commit یا log نشود
`Sms:MeliPayamakOtp:TimeoutSeconds`	`10` ثانیه
`Identity:Session:LifetimeSeconds`	`2592000` ، برابر ۳۰ روز
`Storage:RootPath`	در Development برابر `%LOCALAPPDATA%\MakanApp\Storage`
`Storage:MaxFileSizeBytes`	`10485760` ، برابر ۱۰ MiB
`Storage:UnattachedLifetimeHours`	`24` برای فایل هنوز متصل‌نشده
`Storage:AllowedContentTypes`	`application/pdf` ، `image/jpeg` ، `image/png` ، `text/plain`
`Messaging:MaximumTextLength`	`4000` ؛ بیشتر از سقف مدل قابل تنظیم نیست
`Messaging:DefaultHistoryLimit` / `MaximumHistoryLimit`	`50` / `100`
`Messaging:MaximumAttachmentsPerMessage`	`10`
`Messaging:MaximumMentionsPerMessage`	`50`
`Messaging:DefaultChangeLimit` / `MaximumChangeLimit`	`100` / `200`
`Messaging:MaximumSearchQueryLength`	`200`
`Messaging:DefaultSearchLimit` / `MaximumSearchLimit`	`20` / `50`

مقادیر انتهای جدول که در JSON نوشته نشده‌اند، fallback کد DI هستند. محدودیت IP درخواست OTP هم در `Program.cs` برابر ۱۰ درخواست در دقیقه است؛ این محدودیت درون‌حافظه‌ای و برای یک instance است.

کلید امنیت یا ServiceToken را در مخزن یا سند ذخیره نکنید. اگر provider محلی توسعه بدون کلید ثابت اجرا شود، توکن نشست و challenge OTP قبلی بعد از restart دیگر معتبر نیست. انتخاب `MeliPayamakOtp` بدون `Identity:SecurityKey` پایدار در startup رد می‌شود. appsettings پایه `Sms:Provider=Unavailable` دارد؛ برای OTP واقعی، provider و ServiceToken و کلید پایدار از secret store تنظیم شوند. در Production، `Storage:RootPath` غیرخالی `LocalFileStorage` را فعال می‌کند؛ مسیر باید مطلق، پایدار و قابل نوشتن باشد. مسیر خالی `UnavailableFileStorage` می‌دهد.

قرارداد مشترک HTTP

به‌جز درخواست و تأیید OTP، تمام Endpointهای Controllerها نیازمند نشست معتبر هستند:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
Accept: application/json
```

توکن `accessToken` یک مقدار تصادفی opaque است و JWT نیست. از decode کردن آن برای استخراج نقش استفاده نکنید؛ context معتبر را از `/workspaces/current` بگیرید. Endpoint مستقل refresh token وجود ندارد؛ ورود مجدد از OTP انجام می‌شود.

- نام فیلدهای JSON به شکل `camelCase` است؛ Enumها را به شکل رشته‌های تعریف‌شده مثل `Student` یا `Text` ارسال کنید.
- شناسه‌ها `Guid` هستند. مقدارهای `<...>` در نمونه‌های JSON جایگزین‌شونده‌اند و باید از پاسخ واقعی پر شوند.
- زمان‌های لحظه‌ای را به UTC و با پسوند `Z`، تاریخ‌ها را `yyyy-MM-dd` و ساعت محلی برنامه را `HH:mm:ss` ارسال کنید.
- پاسخ عادی envelope عمومی مثل `data` ندارد؛ همان شیء یا آرایه قرارداد برمی‌گردد. دانلود فایل پاسخ باینری است.
- روش `PATCH` در این پروژه JSON Patch نیست؛ بدنه کامل Command مربوط را ارسال کنید.
- نسخه‌هایی مانند `rowVersion` رشته Base64 هستند. مقدار را همان‌طور که دریافت کرده‌اید در فیلد `expected...` بعدی بفرستید. این APIها قرارداد عمومی `If-Match` ندارند.
- برای نسخه یا cursor داخل Query از URL encoding استفاده کنید؛ مخصوصاً `+` و `/` در Base64 نباید خراب شوند.
- مجوز به انتخاب workspace، عضویت، نقش و رابطه با منبع وابسته است. ارسال `OrganizationId` یا `ClassId` مجوز ایجاد نمی‌کند.

استفاده در PowerShell و Postman

برای درخواست‌های JSON در PowerShell، این helper محلی را می‌توانید در همان نشست تعریف کنید. ارسال بایت UTF-8 متن فارسی را حفظ می‌کند:

```
$apiBase = 'https://localhost:7063/api/v1'
```

```
function Invoke-MakanApi {
    param(
        [string]$Method,
        [string]$Path,
        [object]$Body,
        [string]$Token
    )

    $requestOptions = @{
        Method = $Method
        Uri = "$apiBase$Path"
        Headers = @{ Accept = 'application/json' }
    }
    if ($Token) {
        $requestOptions.Headers.Authorization = "Bearer $Token"
    }
    if ($null -ne $Body) {
        $jsonBody = $Body | ConvertTo-Json -Depth 12 -Compress
        $requestOptions.ContentType = 'application/json; charset=utf-8'
        $requestOptions.Body = [System.Text.Encoding]::UTF8.GetBytes($jsonBody)
    }
    Invoke-RestMethod @requestOptions
}
```

در Postman، سند `/openapi/v1.json` را Import کنید و Authorization مجموعه را روی `Bearer Token` با مقدار توکن جاری بگذارید. در تنظیم فعلی OpenAPI، security scheme اختصاصی Bearer اضافه نشده است؛ وجود Swagger UI را به معنی آماده‌بودن دکمه Authorize برای نشست‌ها در نظر نگیرید. توکن‌های واقعی را در collection یا environment اشتراکی صادر نکنید.

ورود و پروفایل

دریافت OTP و نشست

```
$phoneNumber = Read-Host 'شماره موبایل حساب آزمایشی'
$challenge = Invoke-MakanApi -Method POST -Path '/auth/otp/challenges' -Body @{
    phoneNumber = $phoneNumber
}
```

پاسخ `202` شامل `challengeId`، `expiresAtUtc` و `resendAvailableAtUtc` است؛ کد در پاسخ نیست.

محدودیت ورود دستی در توسعه: `DevelopmentOtpProvider` پیامک نمی‌فرستد و کد تولیدشده را فقط در `DevelopmentOtpStore` همان فرایند نگه می‌دارد. API عمومی برای دریافت کد و OTP ثابت وجود ندارد. برای آزمایش دستی، برنامه را با debugger محلی اجرا کنید و نتیجه `DevelopmentOtpProvider.RequestOtpAsync` را برای همان شماره ببینید؛ آن را در log، فایل یا Git ثبت نکنید. تست خودکار از provider ضبط‌کننده استفاده می‌کند. در محیط `MeliPayamakOtp`، provider کد را تولید و از طریق پیامک ارسال می‌کند و backend فقط hash آن را نگه می‌دارد.

```
$otpCode = Read-Host 'کد دریافت‌شده برای همین challenge'
$login = Invoke-MakanApi -Method POST -Path '/auth/otp/verify' -Body @{
    challengeId = $challenge.challengeId
    phoneNumber = $phoneNumber
    code = $otpCode
}
$accessToken = $login.accessToken
$otpCode = $null
$me = Invoke-MakanApi -Method GET -Path '/me' -Token $accessToken
```

پاسخ تأیید `200` شامل `accessToken`، `tokenType`، `expiresAtUtc` و `user` است. `challengeId` و شماره باید متعلق به همان درخواست OTP باشند؛ درخواست challenge جدید، challenge قبلی را supersede می‌کند. کد provider به‌صورت string opaque و در بازه طول تنظیم‌شده پذیرفته می‌شود؛ اگر کد عددی باشد، ارقام انگلیسی، فارسی و عربی پیش از hash به شکل یکسان normalize می‌شوند. OTP مصرف‌شده قابل استفاده مجدد نیست و تکرار تأیید می‌تواند `OTP_ALREADY_USED` بدهد. اگر پاسخ ورود در شبکه گم شد و توکن را ندارید، مسیر ورود جدید را با رعایت محدودیت ارسال طی کنید.

تکمیل پروفایل و خروج

```
$me = Invoke-MakanApi -Method PATCH -Path '/me/profile' -Token $accessToken -Body @{
    firstName = 'کاربر'
    lastName = 'آزمایشی'
    displayName = 'کاربر آزمایشی'
    username = 'demo.user'
}
```

نام و نام خانوادگی اجباری و هرکدام حداکثر ۱۰۰ کاراکترند. `displayName` اختیاری و حداکثر ۲۰۰ کاراکتر است. `username` باید ۳ تا ۳۲ کاراکتر و شامل حروف لاتین، رقم، نقطه یا زیرخط باشد؛ یکتایی آن نسبت به بزرگی و کوچکی حروف حساس نیست. مقدار نمونه را برای هر حساب تغییر دهید.

`POST /auth/logout` بدون بدنه، نشست جاری را revoke می‌کند و `204` می‌دهد. پس از موفقیت، توکن محلی را پاک کنید.

فضای کاری و سرپرست

شناسه‌ها را با هم اشتباه نگیرید

شناسه	مفهوم و محل مصرف
`UserId`	حساب ورود و طرف گفت‌وگو
`PersonId`	شخص در مدل Identity؛ برابر UserId فرض نشود
`OrganizationPersonId`	رکورد شخص در سازمان؛ ثبت‌نام و انتخاب فرزند از این شناسه استفاده می‌کنند

شناسه	مفهوم و محل مصرف
`MembershipId`	عضویت کاربر در سازمان؛ انتخاب workspace و انتساب معلم
`EnrollmentId`	ثبت‌نام شخص در یک کلاس؛ حضور و غیاب
`SessionId`	در Auth نشست ورود، اما در `/academic/sessions` شناسه جلسه آموزشی است
`ExamAttemptId`	تلاش دانش‌آموز در آزمون
`AttemptQuestionId`	سؤال منجمدشده در یک تلاش؛ برای ذخیره پاسخ، با شناسه سؤال editor جایگزین نشود

انتخاب فضای شخصی یا سازمانی

```
$workspaces = Invoke-MakanApi -Method GET -Path '/workspaces/me' -Token $accessToken
$personalContext = Invoke-MakanApi -Method POST -Path '/workspaces/select' -Token $accessToken -Body @{
    workspaceType = 'Personal'
    membershipId = $null
    role = $null
}
```

برای فضای سازمانی، `membershipId` و `role` را از همان عضو فهرست `/workspaces/me` بگیرید:

```
{
  "workspaceType": "Organization",
  "membershipId": "<membershipId از فهرست فضاها>",
  "role": "Teacher"
}
```

نتیجه `AccessContext` شامل `actorId`، `userId`، `sessionId`، `workspaceType`، `organizationId`، `membershipId`، `activeRole` و `subjectOrganizationPersonId` است. انتخاب در نشست ذخیره می‌شود؛ تغییر آن در یک tab روی درخواست‌های tab دیگری که همان توکن را دارد هم اثر دارد. بعد از تغییر فضا، داده‌های context قبلی را دوباره از API مربوط دریافت کنید.

کاربر بدون عضویت، فضای شخصی دارد. ساخت حساب و تکمیل پروفایل به‌خودی‌خود نقش `Manager`، عضویت سازمانی، رابطه سرپرست یا مجوز تماس شخصی ایجاد نمی‌کند.

دعوت و فرزند

1. با `GET /invitations` دعوت‌های حساب جاری را بگیرید.
2. `POST /invitations/{invitationId}/accept` یا `/decline` را بدون بدنه بفرستید.
3. پس از پذیرش، فهرست workspaceها را تازه کنید و عضویت/نقش موردنظر را انتخاب کنید. پذیرش تکراری همان دعوت می‌تواند با `alreadyAccepted=true` نتیجه قبلی را بدهد.

4. برای والد، ابتدا فضای سازمانی با نقش دقیق `Parent` را انتخاب کنید؛ مقدار `Guardian` نام نقش API نیست.
5. `GET /guardian/children` را بخوانید و `learnerOrganizationPersonId` مجاز را به `POST /guardian/children/{organizationPersonId}/select` بدهید.
6. `GET /workspaces/current` اکنون context فرزند را نشان می‌دهد. رابطه فعال در درخواست‌های بعدی دوباره بررسی می‌شود.

ایجاد سازمان، عضویت اولیه، دعوت، شخص سازمانی و رابطه سرپرست API عمومی ندارند. برای اجرای سناریو سازمانی به داده از قبل آماده‌شده در محیط مجاز نیاز دارید. fixtureهای Integration الگوی داده آزمایشی‌اند و ابزار seed دیتابیس عادی نیستند.

آموزش و حضور و غیاب

مجوزهای اصلی

عملیات	بافت مجاز
ساخت دوره، درس، کلاس، ثبت‌نام و انتساب معلم	`Manager` سازمان انتخاب‌شده
فهرست کلاس‌ها	Manager سازمان، Teacher منتسب، Student ثبت‌نام‌شده؛ Parent در این Endpoint پشتیبانی نمی‌شود
مدیریت جلسه و برنامه، ثبت و اصلاح حضور و غیاب	Manager یا Teacher دارای انتساب فعال به کلاس
برنامه و جزئیات جلسه	نقش مجاز مرتبط با کلاس؛ Parent با فرزند انتخاب‌شده و رابطه معتبر نیز پشتیبانی می‌شود
مشاهده دفتر حضور و غیاب کلاس	Manager یا Teacher منتسب
مدیریت تکلیف و آزمون، ارزیابی و انتشار	Manager یا Teacher منتسب به کلاس
تحویل تکلیف و پاسخ آزمون	Student واجد شرایط و مالک تلاش
نتیجه والد	Parent و context فرزند مجاز؛ DTO مخصوص والد

همه نقش‌های جدول سازمانی‌اند و فقط در همان سازمان و روابط معتبر معنی دارند.

ساخت کلاس

با توکن مدیری که workspace سازمانی را انتخاب کرده است:

```
$period = Invoke-MakanApi -Method POST -Path '/academic/periods' -Token $accessToken -Body @{
    title = 'دوره آزمایشی'
    startDate = '2026-10-01'
    endDate = '2027-02-01'
}
```

```
$course = Invoke-MakanApi -Method POST -Path '/academic/courses' -Token $accessToken -Body @{
    title = 'ریاضی'
}
$class = Invoke-MakanApi -Method POST -Path '/academic/classes' -Token $accessToken -Body @{
    academicPeriodId = $period.id
    courseId = $course.id
    title = 'ریاضی مقدماتی'
    capacity = 20
    activateImmediately = $true
}
```

تاریخ‌های نمونه را متناسب با دوره خود انتخاب کنید. شناسه دوره و درس باید از همین سازمان باشند. ظرفیت مثبت است؛ رقابت برای آخرین ظرفیت در transaction SQL Server کنترل می‌شود. برای مسیر معمول استفاده، `activateImmediately=true` بفرستید؛ Endpoint عمومی مستقلی برای فعال‌کردن کلاس Draft وجود ندارد.

عملیات	بدنه
`POST /academic/classes/{classId}/enrollments`	`{"learnerOrganizationPersonId":"<شناسه شخص سازمانی>"}`
`POST /academic/classes/{classId}/teachers`	`{"teacherMembershipId":"<عضویت فعال معلم>"}`
`POST /academic/classes/{classId}/enrollments/{enrollmentId}/end`	`{"finalStatus":"Completed"}` یا `Withdrawn`
`POST /academic/classes/{classId}/teachers/{teacherAssignmentId}/end`	بدون بدنه

ثبت‌نام یا انتساب فعال تکراری خطا می‌دهد؛ این دو عملیات قرارداد عمومی `clientOperationId` ندارند. پایان ثبت‌نام/انتساب، تاریخچه را حذف نمی‌کند.

جلسه و برنامه تکرارشونده

برای `POST /academic/classes/{classId}/sessions` :

```
{
  "title": "جلسه اول",
  "startUtc": "2026-11-01T06:30:00Z",
  "endUtc": "2026-11-01T08:00:00Z",
  "timeZoneId": "Asia/Tehran",
  "meetingUrl": null
}
```

زمان جلسه جدید باید آینده باشد؛ هنگام اجرای مثال، تاریخ را متناسب با ساعت سرور تغییر دهید. پایان بعد از شروع است. تداخل زمانی کلاس و معلم بررسی می‌شود. پاسخ `SessionResult` دارای `rowVersion` است؛ برای ویرایش، علاوه بر فیلدهای بالا `expectedRowVersion` لازم است. لغو یا تکمیل فقط بدنه `{"expectedRowVersion":"<rowVersion>"}` می‌خواهند.

برای `POST /academic/classes/{classId}/schedule-rules` :

```
{
  "localDayOfWeek": "Sunday",
  "localStartTime": "10:00:00",
  "durationMinutes": 90,
  "timeZoneId": "Asia/Tehran",
  "effectiveFrom": "2026-11-01",
  "effectiveUntil": "2026-11-30",
  "sessionTitle": "جلسه هفتگی",
  "meetingUrl": null
}
```

قانون برنامه، روز و ساعت محلی را همراه timezone نگه می‌دارد و جلسه‌های دارای زمان UTC تولید می‌کند. برای ویرایش قانون، `applyFromDate` و `expectedRowVersion` نیز لازم‌اند. برنامه بازه را با `GET /academic/schedule?fromUtc=...&toUtc=...` بخوانید؛ هر دو زمان UTC، پایان بزرگ‌تر از شروع و بازه حداکثر یک سال است.

ثبت و اصلاح حضور و غیاب

ابتدا `GET /academic/sessions/{sessionId}/attendance` را بگیرید. برای `PUT` همان مسیر:

```
{
  "entries": [
    { "enrollmentId": "<enrollmentId از roster>", "status": "Present" }
  ]
}
```

مقادیر ثبت‌کردنی `Present` ، `Absent` ، `Late` و `Excused` هستند. `NotRecorded` وضعیت نمایش ثبت‌نشدن است. ثبت پیش از شروع جلسه و برای جلسه لغوشده مجاز نیست. برای اصلاح رکورد موجود از `PATCH` `/academic/sessions/{sessionId}/attendance/{attendanceId}` با `status` ، `correctionReason` و `expectedRowVersion` استفاده کنید؛ اصلاح تاریخچه جدا دارد.

تکلیف و ارزیابی

```
پاسخ دانش‌آموز -> Draft -> انتشار و تعیین گیرندگان -> Draft ساخت
-> اتصال فایل / ذخیره متن -> Submit -> Draft ارزیابی -> Release -> نتیجه
```

ساخت و انتشار

با توکن Manager یا Teacher منتسب:

```
$assignment = Invoke-MakanApi -Method POST -Path "/academic/classes/$($class.id)/assignments" -Token
$accessToken -Body @{
    title = 'تمرین اول'
    description = 'پاسخ تمرین را ارسال کنید.'
    dueAtUtc = [DateTime]::UtcNow.AddDays(7).ToString('o')
    allowLateSubmission = $false
    maxAttempts = 2
    maxScore = 20
}
$published = Invoke-MakanApi -Method POST -Path "/academic/assignments/$($assignment.id)/publish" -Token
$accessToken -Body @{
    expectedAssignmentRowVersion = $assignment.assignmentRowVersion
    expectedVersionRowVersion = $assignment.versionRowVersion
}
```

پاسخ انتشار شامل `assignment` و `recipientCount` است. `MaxScore` متعلق به نسخه تکلیف است؛ نمره ارزیابی از این سقف تبعیت می‌کند. پیش‌نویس از مسیر `PATCH /academic/assignments/{assignmentId}` قابل ویرایش است و به هر دو نسخه نیاز دارد. انتشار گیرندگان را از ثبت‌نام‌های واجد شرایط همان زمان تعیین می‌کند؛ صرف عضویت Student برای پاسخ کافی نیست.

فهرست را با `GET /academic/classes/{classId}/assignments` یا `GET /academic/assignments?classId=...` بخوانید. نقش و رابطه با کلاس بر داده قابل مشاهده اثر دارند.

پاسخ و تحویل دانش‌آموز

`$studentToken` در مثال باید توکن نشست Student با workspace سازمانی انتخاب‌شده باشد:

```
$attempt = Invoke-MakanApi -Method POST -Path "/academic/assignments/$($assignment.id)/attempts" -Token
$studentToken
$attempt = Invoke-MakanApi -Method PATCH -Path "/academic/submission-attempts/$($attempt.id)/draft" -Token
$studentToken -Body @{
    answerText = 'پاسخ دانش‌آموز'
    expectedRowVersion = $attempt.rowVersion
}
$receipt = Invoke-MakanApi -Method POST -Path "/academic/submission-attempts/$($attempt.id)/submit" -Token
$studentToken -Body @{
    expectedRowVersion = $attempt.rowVersion
}
```

برای پیوست، ابتدا فایل را با همان حساب و scope مناسب آپلود کنید؛ سپس به `/academic/submission-attempts/{attemptId}/attachments` بدنه `fileAssetId` و `expectedRowVersion` بدهید. پس از هر ذخیره یا اتصال/حذف فایل، از `rowVersion` پاسخ جدید استفاده کنید. حذف پیوست، `expectedRowVersion` را در Query می‌گیرد.

ایجاد Draft، تلاش باز موجود را resume می‌کند. ذخیره Draft تحویل رسمی نیست؛ رسید `SubmissionReceipt` موفقیت Submit را با `submittedAtUtc`، `isLate` و وضعیت `Submitted` نشان می‌دهد. سرور مهلت، سیاست تأخیر، تعداد تلاش و آمادگی فایل را کنترل می‌کند. فیلد `contentVisible` را در پاسخ رعایت کنید؛ وجود metadata تلاش به معنی مجازبودن نمایش محتوای آن نیست.

ارزیابی و انتشار نمره

1. ارزیاب مجاز از `/academic/evaluations/queue` ، با فیلترهای اختیاری `assignmentId` ، `classId` ، `reviewStatus` و `isLate` ، صف را می‌خواند.
2. محتوای پاسخ نهایی از `/academic/submission-attempts/{attemptId}/evaluation/submission` خوانده می‌شود.
3. `PUT /academic/submission-attempts/{attemptId}/evaluation` یک Draft می‌سازد یا ویرایش می‌کند:

```
{
  "score": 18,
  "learnerFeedback": "پاسخ خوب است؛ محاسبه آخر را بررسی کن.",
  "guardianVisibleFeedback": "پیشرفت مناسبی داشته است.",
  "teacherPrivateNote": "یادداشت داخلی ارزیاب",
  "expectedRowVersion": null
}
```

در ایجاد اولیه، `expectedRowVersion=null` است؛ در ویرایش باید نسخه فعلی را بفرستید. نمره در بازه صفر تا `maxScore` نسخه تکلیف است. برای انتشار، به `/evaluation/release` مقدار `expectedRowVersion` پاسخ ارزیابی را بدهید.

دانش‌آموز `/academic/submission-attempts/{attemptId}/result` و والد `/guardian-result` را می‌خوانند. قبل از انتشار، نتیجه تکلیف می‌تواند `404 GRADE_NOT_RELEASED` بدهد. `teacherPrivateNote` به هیچ‌کدام داده نمی‌شود. اصلاح نتیجه منتشرشده از `/evaluation/corrections` با `correctionReason` و `expectedReleasedEvaluationRowVersion` یک Draft تازه می‌سازد؛ انتشار آن نیز جداگانه است.

آزمون و انتشار نمره

طراحی و نسخه‌بندی آزمون

با Manager یا Teacher مجاز، `POST /academic/classes/{classId}/exams` :

```
{
  "title": "آزمون ریاضی",
  "description": "آزمون کوتاه فصل اول",
  "availableFromUtc": "2026-11-01T06:30:00Z",
  "availableUntilUtc": "2026-11-01T08:30:00Z",
  "durationMinutes": 30,
  "maxAttempts": 1,
  "maxScore": 20,
  "randomizationPolicy": "QuestionOrder"
}
```

بازه نمونه را برای زمان اجرای خود تنظیم کنید. سیاست ترتیب سؤال `None` یا `QuestionOrder` است. پاسخ `ExamEditorDto` شامل `examRowVersion` ، `versionRowVersion` و سؤال‌هاست. نمونه افزودن یک سؤال ۲۰ نمره‌ای به `POST` `/academic/exams/{examId}/questions`:

```
{
  "order": 1,
  "type": "ObjectiveSingleChoice",
  "prompt": "حاصل ۲ + ۲ کدام است؟",
  "score": 20,
  "options": [
    { "order": 1, "text": "۴", "isCorrect": true },
    { "order": 2, "text": "۵", "isCorrect": false }
  ],
  "expectedVersionRowVersion": "<versionRowVersion فعلی>"
}
```

سؤال تستی حداقل دو گزینه و دقیقاً یک گزینه درست می‌خواهد؛ ترتیب و متن گزینه‌ها نباید تکراری باشد. سؤال `Descriptive` گزینه ندارد. هنگام انتشار، آزمون باید حداقل یک سؤال داشته باشد و مجموع نمره سؤال‌ها دقیقاً برابر `maxScore` شود.

به `POST /academic/exams/{examId}/publish` دو فیلد `expectedExamRowVersion` و `expectedVersionRowVersion` را از آخرین پاسخ بدهید. نسخه منتشرشده درجا ویرایش نمی‌شود؛ `/versions` با `expectedExamRowVersion` و `expectedPublishedVersionRowVersion` نسخه بعدی می‌سازد. حذف سؤال، برخلاف بسیاری از DELETEها، بدنه JSON با نسخه سؤال و نسخه آزمون دارد.

`GET /academic/exams` فهرست را بر اساس نقش/کلاس یا فرزند مجاز می‌دهد. `/editor` و `/teacher-preview` مخصوص ارزیاب مجازند و اطلاعات پاسخ صحیح دارند. `/student-preview` برای Teacher/Manager مجاز، نمایش امن بدون پاسخ صحیح می‌سازد؛ Student فقط با ثبت‌نام فعال و در بازه فعال آزمون می‌تواند آن را بگیرد. Parent اجازه دیدن محتوای سؤال از این مسیر ندارد.

شروع، ذخیره پاسخ و نهایی‌سازی

```
Student workspace -> Start -> دریافت سؤال‌های منجمد و deadline سرور
-> دریافت write lease -> Save Answer -> دریافت رسید هر ذخیره
-> دریافت رسید نهایی -> Finalize -> توقف ذخیره‌های در حال ارسال
```

1. به `POST /academic/exams/{examId}/attempts/start` بدنه `{"clientOperationId":"<Guid جدید>"}` بفرستید. برای retry همان عملیات، همین Guid را نگه دارید.
2. پاسخ دارای `examAttemptId`، `examVersionId`، `effectiveDeadlineUtc`، `serverNowUtc` و سؤال‌های دارای `attemptQuestionId` است. deadline از محدودترِ پایان بازه آزمون و مدت تلاش محاسبه می‌شود. ترتیب سؤال‌ها برای همان تلاش ثابت می‌ماند.
3. `POST /academic/exam-attempts/{attemptId}/write-lease` بدون بدنه بفرستید. `writeLeaseVersion` برگشتی را نگه دارید. حق نوشتن به نشست تعلق دارد؛ انتقال صریح از `/write-lease/transfer` نسخه را عوض می‌کند و نویسنده قبلی دیگر معتبر نیست.
4. برای هر سؤال، به `PUT /academic/exam-attempts/{attemptId}/answers/{attemptQuestionId}` بدنه زیر را بدهید:

```
{
  "clientOperationId": "<Guid مستقل این ذخیره>",
  "writeLeaseVersion": 1,
```

```
  "expectedRevisionNumber": null,
  "selectedOptionId": "<optionId همان سؤال در تلاش>",
  "textAnswer": null
}
```

`writeLeaseVersion=1` فقط نمونه است؛ مقدار واقعی پاسخ lease را استفاده کنید. در اولین پاسخ `expectedRevisionNumber=null` و در ویرایش، `revisionNumber` آخرین پاسخ پذیرفته‌شده لازم است. برای سؤال تشریحی `selectedOptionId=null` و `textAnswer` متن پاسخ است. رسید ذخیره، `revisionNumber`، `acceptedAtUtc` و `answerSetVersion` جدید را می‌دهد.

برای ادامه پس از reconnect یا تعارض، `GET /academic/exam-attempts/{attemptId}/answers` را بگیرید. این پاسخ مجموعه پاسخ پذیرفته‌شده، نسخه مجموعه و وضعیت write lease را نشان می‌دهد. ذخیره محلی کلاینت یا موفقیت HTTP ارسال‌نشده، اثبات پذیرش پاسخ در سرور نیست.

1. قبل از Finalize، تمام ذخیره‌های در حال ارسال را تمام کنید و نسخه نهایی مجموعه را بگیرید؛ سپس `POST` `/academic/exam-attempts/{attemptId}/finalize`:

```
{
  "clientOperationId": "<Guid مستقل نهایی‌سازی>",
  "expectedAnswerSetVersion": 1,
  "writeLeaseVersion": 1
}
```

هر دو نسخه نمونه‌اند و باید از پاسخ‌های جاری گرفته شوند. رسید شامل `finalizedAtUtc`، `finalizedAnswerSetVersion`، `answeredQuestionCount`، `totalQuestionCount` و `status=Finalized` است. `GET /academic/exam-attempts/{attemptId}/receipt` آن را دوباره می‌دهد. رسید، نمره یا پاسخ صحیح ندارد.

پس از deadline، ذخیره یا Finalize جدید رد می‌شود و تلاش می‌تواند `Expired` شود. **auto-finalize یا scheduler پایان آزمون وجود ندارد**؛ وجود پاسخ ذخیره‌شده، تلاش Expired را به پاسخ تحویل‌شده تبدیل نمی‌کند. retry دقیق Finalize موفق با همان شناسه و payload، رسید قبلی را برمی‌گرداند؛ payload متفاوت با همان شناسه conflict است.

تصحیح و انتشار نمره آزمون

فقط تلاش `Finalized` وارد تصحیح می‌شود؛ منبع تصحیح `ExamFinalAnswers` و revisionهای منجمدشده است.

1. `GET /academic/exams/{examId}/grading` صف تصحیح را با فیلتر اختیاری `classId` و `status` می‌دهد.
2. `POST /academic/exam-attempts/{attemptId}/grading` بدون بدنه، Draft تصحیح را ایجاد یا بازیابی می‌کند. `GET` همان مسیر فقط Draft موجود را می‌خواند.
3. سؤال تستی بر مبنای کلید نسخه منجمد تصحیح می‌شود. سؤال تشریحی به بررسی دستی نیاز دارد. مسیر `PATCH` `/academic/exam-attempts/{attemptId}/grading/questions/{attemptQuestionId}` این بدنه را می‌گیرد:

```
{
  "awardedScore": 15,
  "learnerFeedback": "استدلال را کامل‌تر بنویس.",
  "evaluatorPrivateNote": "یادداشت خصوصی مصحح",
```

```
  "expectedGradeRowVersion": "<rowVersion فعلی تصحیح>"
}
```

نمره هر سؤال باید در بازه مجاز همان سؤال باشد. `expectedGradeRowVersion` نسخه کل Draft نمره است؛ پس از تغییر هر سؤال آن را از پاسخ تازه کنید.

1. به `/grading/complete` فیلدهای `learnerFeedback` ، `guardianVisibleFeedback` ، `evaluatorPrivateNote` و `expectedGradeRowVersion` بدهید؛ بررسی‌های لازم باید کامل شده باشند. وضعیت به `ReadyForRelease` می‌رسد.
2. به `/grade/release` بدنه `clientOperationId` و `expectedGradeRowVersion` بدهید. این مرحله انتشار رسمی است.
3. دانش‌آموز `/result` و والد `/guardian-result` را زیر `/academic/exam-attempts/{attemptId}` می‌خوانند. پیش از انتشار، برای تلاش نهایی‌شده پاسخ `200` با `releaseStatus=AwaitingGrading` یا `AwaitingRelease` و نمره `null` ممکن است؛ این با قرارداد نتیجه تکلیف متفاوت است.
4. `/grade/corrections` با `correctionReason` و `expectedReleasedGradeRowVersion` Draft اصلاحی می‌سازد؛ آن را دوباره کامل و منتشر کنید. تا انتشار اصلاحیه، نتیجه منتشرشده قبلی معتبر می‌ماند.

DTO دانش‌آموز فقط بازخورد دانش‌آموز و DTO والد فقط بازخورد والد را دارد؛ یادداشت خصوصی مصحح و کلید پاسخ در نتیجه عمومی قرار نمی‌گیرند.

فایل‌ها

بارگذاری فایل از `POST /files` با `multipart/form-data` انجام می‌شود؛ نام فیلد `File` است. در Postman بخش Body را روی form-data بگذارید، نوع `File` را برای آن فیلد انتخاب کنید و Content-Type دارای boundary را به خود ابزار بسپارید.

```
$uploadPath = Read-Host 'مسیر فایل PDF آزمایشی'
curl.exe --request POST "$apiBase/files" --header "Authorization: Bearer $accessToken" --form
"File=@$uploadPath;type=application/pdf"
```

پاسخ `201` شامل `id` ، `status` ، `contentType` ، `sizeBytes` ، `sha256Hash` ، `unattachedExpiresAtUtc` و `rowVersion` است. `id` را برای اتصال به تکلیف یا پیام نگه دارید؛ فقط فایل آماده و مجاز قابل اتصال است.

عملیات	مسیر و نکته
metadata	`GET /files/{fileId}`
دانلود	`GET /files/{fileId}/content` با Bearer؛ پاسخ stream است
حذف	`DELETE /files/{fileId}?expectedRowVersion=...` با نسخه URL-encoded

فایل لینک عمومی ندارد. مالکیت، scope و binding به پیام/پاسخ در مجوز دانلود اثر دارند. حذف فایل retained یا در حال استفاده می‌تواند `FILE_IN_USE` بدهد. مهلت فایل unattached به معنی وجود پاک‌سازی پس‌زمینه زمان‌بندی‌شده نیست.

در تنظیم پیش‌فرض فقط PNG، JPEG، PDF و متن مجازند. مدل Messaging نوع `Video` و `Voice` را می‌شناسد، اما برای آپلود آن‌ها باید MIME مجاز مناسب در تنظیم محیط وجود داشته باشد؛ تنظیم پایه این نوع‌ها را نمی‌پذیرد. اعتبارسنجی MIME و اندازه، جای اسکن بدافزار نیست؛ اسکن و object storage تولیدی در این نسخه ارائه نشده‌اند.

پیام‌رسانی و همگام‌سازی

آغاز گفت‌وگو و مجوز ارتباط

`POST /conversations/direct` :

```
{
  "targetUserId": "<UserId مخاطب مجاز>",
  "scope": "Organization"
}
```

Scope یکی از `Personal` یا `Organization` است. در Scope سازمانی، سازمان از workspace جاری به دست می‌آید. ایجاد گفت‌وگوی تازه `201` و بازیابی زوج موجود `200` با `alreadyExisted=true` دارد.

در فضای شخصی، شروع تماس به دو حساب با وضعیت سنی `Adult` و `PersonalCommunicationGrant` فعال نیاز دارد؛ ثبت‌نام ساده این شرایط را تأمین نمی‌کند. برای وضعیت سنی و ایجاد grant، API عمومی وجود ندارد.

در فضای سازمانی، عضویت‌ها و نقش‌ها فعال‌اند و policy فعلی این روابط را کنترل می‌کند:

نقش آغازگر	مخاطب‌های مجاز با شرایط لازم
Manager	Parent، Teacher، Manager و Student با ثبت‌نام فعال
Teacher	Manager و Teacher؛ Student کلاس مرتبط؛ Parent با رابطه آموزشی معتبر
Student	با ثبت‌نام فعال: Manager و Teacher مرتبط
Parent	Manager و Teacher مرتبط با فرزند

شناسه مخاطب ناشناخته یا غیرمجاز می‌تواند خطای عمومی `DIRECT_RECIPIENT_NOT_AVAILABLE` بدهد. API عمومی جست‌وجوی کاربران با شماره تلفن وجود ندارد.

گروه، کانال و پیام

برای ایجاد گروه یا کانال از `/conversations/groups` یا `/conversations/channels` استفاده کنید:

```
{
  "clientOperationId": "<Guid ایجاد گفت‌وگو>",
  "scope": "Organization",
```

```
  "title": "گروه آموزشی",
  "description": "هماهنگی کلاس",
  "initialParticipantUserIds": []
}
```

ایجاد گروه/کانال سازمانی به workspace معتبر با نقش Manager یا Teacher نیاز دارد؛ ایجاد شخصی به workspace شخصی و وضعیت سنی Adult. افزودن عضو نیز policy ارتباط با مخاطب را طی می‌کند. نقش‌های داخل گفت‌وگو `Owner`، `Admin` و `Member` از نقش‌های سازمانی جدا هستند. در Group اعضای فعال مجاز به ارسال‌اند؛ در Channel انتشار به Owner/Admin محدود است. مالک باید پیش از خروج، مالکیت را با جریان درخواست انتقال و پذیرش مقصد واگذار کند. archive با حذف تاریخچه یکی نیست.

برای ارسال `POST /conversations/{conversationId}/messages`:

```
{
  "clientMessageId": "<Guid این پیام منطقی>",
  "kind": "Text",
  "text": "سلام، زمان جلسه تأیید شد.",
  "attachmentIds": [],
  "replyToMessageId": null,
  "mentionedUserIds": []
}
```

رسید `200` دارای `messageId`، `sequence`، `sentAtUtc`، `status=Sent` و `version` است. `Sent` یعنی commit موفق در SQL Server؛ دریافت یا خواندن توسط مخاطب را تضمین نمی‌کند. همان `clientMessageId` را برای retry همان پیام نگه دارید؛ تغییر payload با شناسه قبلی `MESSAGE_IDEMPOTENCY_CONFLICT` می‌دهد.

برای پیام رسانه‌ای `kind` مناسب و `attachmentIds` فایل‌های آماده لازم است. Reply با شناسه پیام همان گفت‌وگو، Mention با UserId عضو مجاز و Forward با `destinationConversationId` و یک `clientMessageId` جدید انجام می‌شود؛ دسترسی به هر دو طرف و محدودیت Scope کنترل می‌شود.

ویرایش پیام `text` و `expectedVersion` می‌گیرد؛ حذف پیام `expectedVersion` در Query می‌خواهد. حذف یک tombstone ایجاد می‌کند؛ در نمایش `isDeleted` و محتوای پنهان‌شده را رعایت کنید. واکنش‌های موجود `Like`، `Love`، `Laugh`، `Wow` و `Sad` هستند. مسیرهای Pin، اعضا، انتقال مالکیت و رسانه در مرجع Endpointها آمده‌اند.

تاریخچه، Delta و رسید خواندن

نیاز	درخواست یا پاسخ
فهرست گفت‌وگوها	`GET /conversations`؛ دارای `unreadCount` و cursorهای Read/Delivered
تاریخچه	`GET /conversations/{id}/messages?limit=50`
صفحه قدیمی‌تر	مقدار `nextBeforeSequence` صفحه قبلی را در `beforeSequence` بفرستید

نیاز	درخواست یا پاسخ
تغییرات	`GET /conversations/{id}/changes?afterCursor=...&limit=100`
صفحه بعد تغییرات	`nextCursor` را حفظ کنید و تا `hasMore=false` ادامه دهید
اعلام دریافت	`POST /conversations/{id}/delivered` با `{"upToMessageSequence":12}`
اعلام خواندن	`POST /conversations/{id}/read` با همان شکل بدنه

عدد `12` نمونه است؛ فقط آخرین Sequence واقعاً دریافت‌شده یا خوانده‌شده را ACK کنید. cursor خواندن/دریافت عقب نمی‌رود. `MessageSequence` ترتیب پیام‌ها و cursor تغییرات ترتیب mutationها را بیان می‌کنند؛ این دو قابل جایگزینی نیستند. cursorها را opaque نگه دارید و به گفت‌وگو/جست‌وجوی دیگری منتقل نکنید.

SignalR

Hub مسیر `/hubs/messaging` دارد. متدهای قابل فراخوانی `SubscribeConversation(conversationId)` و `UnsubscribeConversation(conversationId)` هستند. event سمت کلاینت `ConversationChanged` نام دارد و اطلاعات سبک مثل `conversationId` ، `type` ، `resourceId` ، `resourceVersion` ، `cursor` ، `occurredAtUtc` و `payloadVersion` را حمل می‌کند؛ متن کامل پیام نیست.

پس از اتصال یا reconnect، دسترسی و subscriptionها را برقرار کنید و تغییرات را از آخرین cursor پردازش‌شده با REST بگیرید. صرف دریافت یک اعلان یا cursor جدید، مجوز ردکردن تغییرات بین cursor قبلی و آن اعلان نیست. بعد از پردازش موفق صفحه cursor ،Delta ذخیره‌شده را جلو ببرید. تحویل اعلان می‌تواند تکراری باشد؛ `changeId` را برای تشخیص تغییر تکراری نگه دارید.

محدودیت فعلی احراز هویت Hub: `SessionAuthenticationHandler` فقط `Authorization: Bearer ...` را می‌خواند و `access_token` در query را نمی‌خواند. CORS اکنون برای همه origin/header/methodها باز و بدون `AllowCredentials` است؛ این تغییر پشتیبانی query-token WebSocket/SSE مرورگر را اضافه نمی‌کند. REST مرورگر از `credentials: "omit"` و SignalR LongPolling از `accessTokenFactory` و `withCredentials:false` استفاده کند؛ نمونه در قرارداد فرانت‌اند است. transport و reconnect در origin واقعی باید آزمایش شوند. در قطع REST ،realtime و Delta مسیر بازیابی‌اند؛ backplane و cookie-based affinity از تنظیم فعلی قابل نتیجه‌گیری نیستند.

جست‌وجو، مسدودسازی و گزارش

`GET /messages/search?q=...` از فیلترهای اختیاری `conversationId` ، `kind` ، `fromUtc` ، `toUtc` ، `cursor` و `limit` پشتیبانی می‌کند. پاسخ شامل `items` ، `totalCount` و `nextCursor` است. Query و cursor را URL-encode کنید؛ نتایج بر اساس دسترسی جاری محدود می‌شوند.

- `POST /messaging/blocks/{userId}` حساب را مسدود می‌کند؛ `DELETE` همان مسیر رفع مسدودسازی است؛ `GET /messaging/blocks` فهرست خود کاربر را می‌دهد.
- `POST /conversations/{conversationId}/messages/{messageId}/reports` بدنه `clientReportId` ، `reason` و `description` می‌گیرد.
- دلیل گزارش یکی از `Harassment` ، `Spam` ، `Threat` ، `InappropriateContent` ، `Impersonation` یا `Other` است.

- `GET /messaging/reports/{reportId}` رسید گزارش متعلق به خود کاربر را می‌دهد. این API پنل رسیدگی مدیر یا تضمین اقدام خودکار روی گزارش نیست.

خطاها و تلاش مجدد

خطای کاربردی به شکل ProblemDetails برمی‌گردد؛ نمونه:

```
{
  "type": "urn:makan:problem:auth-required",
  "title": "Authentication required",
  "status": 401,
  "detail": "برای ادامه باید وارد شوید.",
  "code": "AUTH_REQUIRED",
  "traceId": "<شناسه پیگیری درخواست>"
}
```

کلاینت تصمیم ماشینی را از status و `code` بگیرد و متن `detail` را برای توضیح به کاربر استفاده کند. خطاهای model binding، JSON نامعتبر یا validation خود ASP.NET Core ممکن است `errors` داشته باشند و `code` کاربردی نداشته باشند؛ مسیر fallback لازم است.

status	نمونه و اقدام
`400`	ورودی نامعتبر، نوع پاسخ یا cursor اشتباه؛ درخواست را اصلاح کنید
`401`	`AUTH_REQUIRED` ؛ توکن منقضی، revoke یا با restart نامعتبر شده است
`403`	نقش، workspace، رابطه آموزشی یا مجوز منبع کافی نیست
`404`	منبع پیدا نشد یا برای جلوگیری از افشای وجود آن پنهان شده؛ شناسه و context را بررسی کنید
`409`	تعارض lifecycle، ظرفیت، مهلت، idempotency یا نسخه پاسخ آزمون؛ `code` را تفکیک کنید
`410`	OTP یا دعوت منقضی/غیرقابل استفاده شده است
`412`	`CONCURRENCY_CONFLICT` یا `MESSAGE_EDIT_CONFLICT` ؛ داده و نسخه جاری را دوباره بگیرید
`413`	حجم فایل بیش از سقف
`415`	MIME فایل مجاز نیست
`422`	`SMS_PROVIDER_REJECTED` ؛ provider درخواست ارسال را رد کرده؛ retry کور انجام نشود

status	نمونه و اقدام
`429`	محدودیت OTP یا تلاش‌ها؛ ارسال خودکار سریع انجام ندهید
`503`	`FILE_STORAGE_FAILED` ، provider پیامک در دسترس نیست یا credential سرویس رد شده؛ خطای نشست کاربر نیست
`500`	خطای غیرمنتظره؛ `traceId` را برای بررسی سرور نگه دارید

قراردادهای retry

عملیات	شناسه یا قاعده
پذیرش دعوت	همان دعوت؛ نتیجه قبلی با `alreadyAccepted` قابل بازیابی است
شروع Direct	زوج کاربران و Scope؛ گفت‌وگوی قبلی resolve می‌شود
ساخت Group/Channel	`clientOperationId`
ارسال یا Forward پیام	`clientMessageId` در محدوده فرستنده و گفت‌وگو
گزارش پیام	`clientReportId`
شروع آزمون	`clientOperationId`
ذخیره پاسخ آزمون	`clientOperationId` هر ذخیره، همراه lease و revision موردانتظار
Finalize آزمون	`clientOperationId` و payload یکسان
انتشار نمره آزمون	`clientOperationId` و payload یکسان
ویرایش‌های مبتنی بر rowversion	پس از conflict ابتدا داده را بخوانید و تغییر را بازبینی کنید

هدر عمومی `Idempotency-Key` پیاده‌سازی نشده است. برای عملیات جدید Guid جدید و برای retry همان عملیات همان Guid و payload را نگه دارید. همه POSTها retry-safe نیستند؛ ساخت دوره، درس، کلاس یا تکلیف را با فرض idempotency تکرار نکنید. کنترل نسخه نیز با idempotency یک مفهوم نیست.

داشبوردهای نقش‌محور

داشبوردهای STEP 11A از داده رسمی موجود در SQL Server ساخته می‌شوند و برای چهار مخاطب قرارداد جدا دارند:

مخاطب	Endpoint	context لازم
Student	`GET /api/v1/dashboards/student`	workspace سازمانی با نقش فعال `Student`

مخاطب	Endpoint	context لازم
Parent	`GET /api/v1/dashboards/parent`	workspace نقش `Parent` و در صورت مشاهده فرزند، child context انتخاب‌شده
Teacher	`GET /api/v1/dashboards/teacher`	workspace نقش `Teacher` و `TeacherAssignment` فعال
Manager	`GET /api/v1/dashboards/manager`	workspace نقش `Manager` همان سازمان

درخواست LearnerId، OrganizationId، body یا Role ندارد. نقش و سازمان از نشست و `AccessContext` سمت سرور resolve می‌شوند. Parent باید از `GET /api/v1/guardian/children` و `POST` `/api/v1/guardian/children/{learnerOrganizationPersonId}/select` موجود استفاده کند؛ dashboard سیستم انتخاب موازی ایجاد نمی‌کند. در نبود child انتخاب‌شده پاسخ Parent موفق اما خالی است و پس از revoke شدن رابطه، درخواست بعدی با `CHILD_CONTEXT_NOT_ALLOWED` متوقف می‌شود.

همه پاسخ‌ها `dataAsOfUtc` ، `organizationId` و `audience` دارند. `dataAsOfUtc` زمان آغاز خواندن روی سرور است و به معنی realtime یا snapshot واحد میان تمام queryها نیست. چون timezone سازمان هنوز مدل نشده است، windowهای داشبورد بر UTC تکیه دارند.

قواعد مهم مصرف پاسخ:

- Student و Parent فقط نمره‌ای را می‌بینند که release رسمی متناظر دارد؛ Draft evaluation یا grade آماده انتشار score ندارد.
- Attendance از رکورد رسمی Attendance می‌آید؛ `notRecordedCount` را Absent نمایش ندهید.
- Exam summary فقط metadata امن دارد و سؤال، option صحیح، answer key و note خصوصی در آن نیست.
- Teacher فقط کلاس‌ها و backlogهای دارای assignment معتبر خودش را می‌بیند.
- Manager summary سازمانی می‌گیرد و محتوای پیام/پاسخ خصوصی در آن وجود ندارد.
- collectionها bounded هستند؛ این API جایگزین گزارش تفصیلی یا pagination مرحله STEP 11B نیست.

نمونه empty state دانش‌آموز:

```
{
  "dataAsOfUtc": "2026-10-11T09:30:00Z",
  "organizationId": "00000000-0000-0000-0000-000000000001",
  "audience": "Student",
  "nextSession": null,
  "nearestAssignmentDeadline": null,
  "recentReleasedFeedback": [],
  "upcomingExams": [],
  "attendanceSummary": {
    "eligibleSessionCount": 0,
    "recordedCount": 0,
    "presentCount": 0,
    "absentCount": 0,
    "lateCount": 0,
    "excusedCount": 0,
```

```
    "notRecordedCount": 0
  },
  "pendingAssignmentCount": 0
}
```

فیلد AI/Copilot در قرارداد ساخته نشده است. insightهای هوشمند به STEP 10 و widgetهای Plan/Quota/Subscription مدیر به STEP 13 Commerce موکول شده‌اند. معماری و جزئیات privacy در سند STEP 11A آمده است.

گزارش‌های تفصیلی نقش‌محور

STEP 11B در کنار داشبوردهای خلاصه، APIهای report خواندنی و صفحه‌بندی‌شده را ارائه می‌کند. Dashboard برای glance سریع است؛ report برای sort، filter، history و drill-down استفاده می‌شود. مسیرهای اصلی:

- Student: `GET /api/v1/reports/student/attendance|assignments|exams|grades`
- Parent: `GET /api/v1/reports/parent/attendance|assignments|exams`
- Teacher: `GET /api/v1/reports/teacher/classes/{classId}/attendance|assignments|exams` و `GET /api/v1/reports/teacher/classes/{classId}/learners/{learnerId}`
- Manager: `GET /api/v1/reports/manager/enrollments|attendance|assessments|teacher-workload` و `GET /api/v1/reports/manager/classes/{classId}`

فهرست‌ها `dataAsOfUtc`، `pageNumber`، `pageSize`، `totalCount` و `items` دارند. پیش‌فرض صفحه ۱ با اندازه ۲۰ و سقف اندازه ۱۰۰ است. `fromUtc` / `toUtc` باید UTC باشند. sort فقط از allowlist هر report پذیرفته می‌شود و مقدار arbitrary با `REPORT_SORT_INVALID` رد می‌شود.

سازمان، نقش Student/Parent/Teacher/Manager و child context از session و `AccessContext` سمت سرور می‌آیند. `classId`، `courseId` و `academicPeriodId` فقط selector هستند. Teacher باید assignment فعال همان کلاس داشته باشد و Parent باید child context معتبر انتخاب کرده باشد. Guid خارج محدوده ممکن است برای جلوگیری از افشای منبع `REPORT_RESOURCE_NOT_FOUND` بدهد.

در گزارش حضور، نبود رکورد رسمی `NotRecorded` است و به `Absent` تبدیل نمی‌شود. پاسخ attendance علاوه بر page، یک `summary` روی کل مجموعه فیلترشده با شمارنده‌های eligible/recorded/status/not-recorded دارد. نمره Student/Parent فقط پس از release رسمی نمایش داده می‌شود. Parent فقط guardian-visible feedback را می‌بیند. گزارش آزمون هیچ سؤال، گزینه صحیح، answer key، پاسخ learner یا note خصوصی ندارد. عملکرد Manager درصد نرمال‌شده `score / maxScore` را برای Assignment و Exam جدا محاسبه می‌کند و سقف سراسری ۲۰ یا GPA فرض نمی‌کند.

نمونه صفحه خالی معتبر:

```
{
  "dataAsOfUtc": "2026-10-11T09:30:00Z",
  "pageNumber": 3,
  "pageSize": 20,
  "totalCount": 0,
```

```
  "items": []
}
```

جزئیات endpointها، allowlist مرتب‌سازی و privacy در سند معماری STEP 11B آمده است. Export، گزارش زمان‌بندی‌شده، cache و AI در این مرحله وجود ندارند.

نقشه persistence و مرز تراکنش‌ها

MakanDbContext از configurationهای همان assembly استفاده می‌کند. ۵۳ جدول در هفت schema وجود دارد؛ Reports داده جدید ذخیره نمی‌کند. برای ستون‌ها، FKها، indexها و CHECKهای کامل، فرهنگ دیتابیس مرجع تکمیلی است.

schema	تعداد جدول	گروه داده
`identity`	۵	User، Person، Credential، OTP و UserSession
`organization`	۵	Organization، OrganizationPerson، Membership، RoleAssignment، Invitation و
`guardian`	۱	GuardianRelation
`academic`	۹	دوره، درس، کلاس، Enrollment، TeacherAssignment، ScheduleRule، Session و تاریخچه حضور
`assessment`	۱۸	تکلیف/نسخه/گیرنده، پاسخ و نمره تکلیف؛ تعریف/تلاش/پاسخ/نمره آزمون
`messaging`	۱۴	گفتگو/عضو/پیام/فایل/واکنش، تغییرات، Report، Block، outbox و مجوز ارتباط
`storage`	۱	FileAsset؛ محتوای باینری در storage adapter است

سه لایه تضمین درستی داده

1. Domain قوانین محلی و تغییر وضعیت entity را کنترل می‌کند؛ مثل معتبر بودن نمره و غیرقابل ویرایش بودن نسخه منتشرشده.
2. Application context و چند منبع را کنار هم بررسی می‌کند؛ مثل معلم مجاز کلاس، enrollment معتبر و موعد تحویل.
3. SQL Server با transaction، CHECK، unique/filtered index، FK و rowversion از invariant پایدار محافظت می‌کند. این لایه برای رقابت دو درخواست ضروری است؛ صرف بررسی سرویس کافی نیست.

FKهای مرکب مانند `(OrganizationId, ClassId)` تطابق بافت را الزام می‌کنند. این FKها حق خواندن یک endpoint را اثبات نمی‌کنند. filtered unique indexها معمولاً یکتایی رابطه فعال را نگه می‌دارند تا سابقه پایان‌یافته حفظ شود. CHECK SQL نیز فقط محدودیت تعریف‌شده خود را تضمین می‌کند؛ lifecycle کامل محصول یک CHECK واحد نیست.

revision، rowversion و operationId سه چیز متفاوت‌اند

ابزار	سؤال پاسخ‌داده‌شده	رفتار مصرف‌کننده
SQL `rowversion`، در API به شکل Base64	آیا ردیف از آخرین خواندن من عوض شده؟	نسخه response جاری را برای write بعدی بفرستید؛ در412 دوباره بخوانید
`expectedRevisionNumber` / `expectedAnswerSetVersion`	آیا مجموعه/پاسخ آزمون همان revision مورد انتظار است؟	receipt آخرین ذخیره پذیرفته‌شده را نگه دارید؛ conflict را با merge/refetch حل کنید
`clientOperationId` / `clientMessageId` / `clientReportId`	آیا این همان عملیات قبلی است که پاسخ آن گم شده؟	در retry همان شناسه و همان payload؛ برای عملیات تازه شناسه تازه
`writeLeaseVersion`	آیا نشست جاری با نسخه درست حق نوشتن روی تلاش را دارد؟	acquire/transfer طبق API؛ داشتن Bearer جای lease را نمی‌گیرد

rowversion زمان رویداد نیست. پارامتر expected version در body/query همان endpoint قرار دارد؛ هدر عمومی If-Match یا Idempotency-Key برای همه APIها وجود ندارد. همه POSTها retry-safe نیستند.

نمونه‌های واقعی تراکنش و locking

عملیات	مرز واقعی	نکته
Verify OTP	`IdentityService` + `EfIdentityStore`، Serializable	مصرف challenge و ساخت account/session در یک عملیات تراکنشی؛ خطای verify نیز وضعیت تلاش ناموفق را ثبت می‌کند
قبول دعوت	`OrganizationService` + `EfOrganizationStore`	Invitation برای حساب مقصد با `UPDLOCK/HOLDLOCK` خوانده و وضعیت مجدداً بررسی می‌شود
ثبت‌نام/پایان ثبت‌نام	`AcademicService` + `EfAcademicStore`، Serializable	قفل کلاس و بازخوانی ظرفیت/رابطه فعال؛ جلوگیری از مصرف آخرین ظرفیت توسط دو درخواست
save answer / finalize	`ExamAnswerService/ExamFinalizationSer` `vice` + store متناظر	قفل تلاش، مجوز مالک، session/lease/deadline و version پیش از write/commit
mutation پیام	`EfMessagingStore` و فایل‌های partial آن	داده پیام، change و outbox در مرز مربوط به mutation پایدار می‌شوند
dispatch اعلان	`MessagingRealtimeOutboxProcessor`	claim در transaction کوتاه؛ تماس notifier پس از commit claim انجام می‌شود

ارسال SMS در `RequestOtpAsync` قبل از ذخیره challenge است و در transaction طولانی SQL انجام نمی‌شود. اگر provider موفق شود ولی ذخیره SQL شکست بخورد، SMS رسیده الزاماً challenge قابل تأیید در backend ندارد. retry کور ارسال پیامک را پیشنهاد نکنید. storage فایل هم transaction مشترک اتمیک با SQL Server ندارد؛ مسیرهای جبران و وضعیت FileAsset در `StorageService` بررسی شوند.

migration و محیط توسعه

۲۰ migration موجود از foundation تا `AddExamGradingAndRelease` مدل فعلی را می‌سازند. Dashboard، Detailed Reports و adapter OTP migration اختصاصی جدید ندارند. runtime هنگام startup یا `Migrate` یا `EnsureCreated` اجرا نمی‌کند. `MakanDbContextFactory` برای ابزار EF از متغیر `ConnectionStrings__MakanDatabase` استفاده می‌کند؛ connection runtime پروژه Api را خودکار از User Secrets نمی‌گیرد.

برای تغییر schema ابتدا entity و configuration را کامل کنید، سپس migration بسازید و فایل‌های Up/Down و snapshot را بررسی کنید. فرمان زیر **الگوی توسعه** است؛ نام migration را متناسب تغییر واقعی انتخاب کنید و مقصد development را صریح تنظیم کرده باشید:

```
dotnet tool restore
dotnet tool run dotnet-ef migrations add DescribeActualSchemaChange --project
MakanApp.Infrastructure/MakanApp.Infrastructure.csproj --startup-project
MakanApp.Infrastructure/MakanApp.Infrastructure.csproj
```

نام نمونه بالا migration موجود نیست. این مرحله مستندسازی migration نساخته است. اجرای migration روی Production یک عملیات جدا با artifact بررسی‌شده و برنامه backup/rollback است؛ سند استقرار مسیر آن را توضیح می‌دهد.

شش مسیر واقعی برای مطالعه کد از ابتدا تا انتها

۱. درخواست و تأیید OTP

```
AuthController.RequestOtp
-> IdentityService.RequestOtpAsync
-> NormalizePhoneNumber + persisted rate/resend checks
-> IOtpDeliveryProvider.RequestOtpAsync
-> MeliPayamakOtpProvider OR DevelopmentOtpProvider
-> normalize returned code + IdentityCryptography.HashOtp
-> OtpChallenge + EfIdentityStore.SaveChangesAsync
-> RequestOtpResult (202)
```

provider مدیریت‌شده کد را تولید می‌کند؛ backend همان کد را hash می‌کند و کد موازی محلی نمی‌سازد. latest challenge پس از پاسخ قابل استفاده provider supersede می‌شود و زمان expiry از زمان بعد از تحویل provider محاسبه می‌شود. در verify، `OtpChallenge.Verify` مصرف/انقضا/تعداد تلاش را تعیین می‌کند؛ credential موجود User را resolve می‌کند یا User تازه ساخته می‌شود، سپس UserSession با hash توکن ساخته می‌شود. token خام فقط در response ساخت نشست تحویل داده می‌شود.

مسیر مطالعه: AuthController → IdentityService → OtpChallenge → IdentityAbstractions → EfIdentityStore → IdentityEndpointsTests.

۲. آخرین ظرفیت کلاس

```
ClassesController.EnrollLearner
-> AcademicService.EnrollLearnerAsync
-> resolve Organization / Manager
-> Serializable transaction -> lock Class
-> validate learner OrganizationPerson + existing active enrollment + capacity
-> Enrollment.CreateActive / store.Add
-> SaveChanges + Commit -> EnrollmentResult
```

هدف قفل کلاس این است که دو درخواست آخرین جای خالی را هم‌زمان آزاد نبینند. یکتایی enrollment فعال در SQL نیز مکمل کنترل سرویس است. پایان enrollment سابقه را حذف نمی‌کند؛ status و زمان پایان ثبت می‌شوند. API ثبت‌نام LearnerOrganizationPersonId می‌گیرد، نه شناسه حساب User.

مسیر مطالعه: ClassesController → AcademicService → Enrollment → EfAcademicStore → EnrollmentConfiguration → AcademicEndpointsTests.

۳. تحویل تکلیف و انتشار نمره

انتشار تکلیف، نسخه و گیرندگان رسمی آن را مشخص می‌کند. Student draft خودش را می‌سازد/بازیابی می‌کند، متن و فایل مجاز را ذخیره می‌کند و FinalSubmit می‌زند. پاسخ موفق تحویل با حفظ clientOperationId قابل بازیابی است؛ ذخیره draft به معنی Submitted نیست. فایل ابتدا upload می‌شود و attach مجاز به submission مرحله جداست.

ارزیاب از `EvaluationService` یک `EvaluationRevision` draft می‌سازد، سپس release رسمی `GradeRelease` را ایجاد می‌کند. Student و Parent projection نتیجه منتشرشده خود را می‌گیرند؛ TeacherPrivateNote در خروجی عمومی قرار نمی‌گیرد. اصلاح نمره منتشرشده revision اصلاحی و دلیل می‌خواهد و سابقه رسمی را بی‌صدا overwrite نمی‌کند.

مسیر مطالعه: SubmissionsController → SubmissionService → SubmissionAttempt → EfSubmissionStore؛ سپس EvaluationService → EfEvaluationStore → EvaluationLifecycleEndpointsTests.

۴. پاسخ آزمون تا نتیجه رسمی

```
ExamService: published version
-> ExamAttemptService: freeze questions/order + server deadline
-> ExamAnswerService: lease + expected revision + accepted AnswerRevision
-> ExamFinalizationService: final snapshot + receipt
-> ExamGradingService: objective scoring + descriptive review
-> ReadyForRelease -> ExamGradeRelease
-> authorized Student/Guardian result
```

هر تلاش به نسخه و مجموعه سؤال خودش وابسته است. زمان دستگاه مرجع مهلت نیست. ذخیره پاسخ، revision جدید قابل پیگیری تولید می‌کند؛ finalize مجموعه پاسخ پذیرفته‌شده را تثبیت می‌کند و receipt دارد. حق نوشتن به نشست مربوط است تا دو دستگاه بی‌صدا پاسخ همدیگر را overwrite نکنند. تصحیح تستی و تکمیل ارزیابی با انتشار عمومی یک مرحله نیستند. وجود deadline به معنی worker نهایی‌سازی خودکار نیست؛ auto-finalize در این نسخه وجود ندارد.

مسیر مطالعه: ExamsController → ExamAttemptService → ExamAnswerService → ExamFinalizationService → ExamGradingService → ExamFinalizationEndpointsTests و ExamGradingEndpointsTests.

۵. ارسال پیام، Delta و SignalR

```
ConversationsController.SendMessage
-> MessagingService + communication/participant policy
-> EfMessagingStore: message + stable change event + outbox
-> commit -> MessageReceiptResult
-> MessagingRealtimeOutboxDispatcher
-> MessagingRealtimeOutboxProcessor -> SignalRMessagingRealtimeNotifier
-> ConversationChanged notification -> client fetches Delta
```

`clientMessageId` اثر retry را محدود می‌کند. receipt HTTP نتیجه commit است؛ SignalR می‌تواند دیرتر برسد یا اتصال قطع باشد. notification یک اشاره به تغییر است، نه payload کامل پیام خصوصی. Delta از `MessagingChangeEvent` مرجع بازیابی تغییرات بعد از cursor است. cursor delivered/read وضعیت خود کاربر است؛ از آن رسید خواندن همه مخاطبان نتیجه نگیرید.

processor در MessagingRealtimeOutboxDispatcher.cs تعریف شده است. claim با `UPDLOCK/READPAST/ROWLOCK`، batch حداکثر۵۰ و lease۳۰ثانیه‌ای انجام می‌شود؛ ارسال notifier خارج transaction claim است. از retry، lease و crash باید امکان ارسال تکراری notification را نتیجه گرفت؛ مصرف‌کننده با شناسه/نسخه و Delta باید تکرار را تحمل کند. backplane چند instance در این نسخه ثبت نشده است.

مسیر مطالعه: EfMessagingStore → MessagingService → ConversationsController و فایل‌های partial → MessagingSyncEndpointsTests → SignalRMessagingRealtimeNotifier → dispatcher/processor.

۶. یک گزارش نقش‌محور

`ReportsController` query را bind می‌کند. `DetailedReportService` context و نقش مسیر، sort allowlist، page size و بازه UTC را بررسی می‌کند. `EfDetailedReportReadService` query/projection را روی جداول رسمی همان scope اجرا می‌کند. تغییر وضعیت entity یا جدول گزارش تازه‌ای لازم نیست. این تفاوت یک read use case با mutation است.

Student/Parent فقط release رسمی نتیجه را می‌بینند. مقدار `dataAsOfUtc` زمان شروع خواندن است و مجموعه چند query را به snapshot اتمیک تبدیل نمی‌کند. برای teacher-workload بازه زمانی فقط completedSessionCount را محدود می‌کند؛ مراجعه به کارت دقیق endpoint برای معنی فیلترها ضروری است.

مسیر مطالعه: ReportsController → DetailedReportService → DetailedReportAbstractions → EfDetailedReportReadService → DetailedReportServiceTests.

روش توسعه و عیب‌یابی برای عضو جدید تیم

یک قابلیت کوچک را چطور اضافه یا اصلاح کنیم؟

1. مسئله، endpoint/رفتار، نقش مجاز و scope منبع را دقیق بنویسید. AGENTS و اسناد مرتبط را بخوانید؛ وضعیت Git و تغییرات از قبل موجود را بررسی کنید.
2. مسیر Controller → service → domain → port → store را پیدا کنید. برای یک قانون خالص، تست Unit؛ برای API/SQL/مجوز/رقابت، تست Integration انتخاب کنید.
3. قرارداد request/result و خطاها را تعیین کنید. قبل از افزودن فیلد actor/organization از کلاینت، بررسی کنید آیا باید از context معتبر سرور resolve شود.
4. قانون را در لایه مناسب تغییر دهید. Controller فقط transport را map کند. I/O و EF در Infrastructure؛ async و CancellationToken تا انتهای عملیات عبور کنند.
5. اگر persistence تغییر کرد، constraints، configuration و migration را کامل و بررسی کنید. برای concurrency، فقط check اولیه ننویسید؛ race condition و transaction/locking را مشخص کنید.
6. retry را بررسی کنید: payload یکسان و operationId یکسان باید چه پاسخ/اثر داشته باشند؟ همان شناسه با payload متفاوت چه خطایی می‌دهد؟ کار شبکه‌ای طولانی داخل transaction نباشد.
7. test، build، restore و validation نهایی را اجرا کنید؛ سپس diff، اسناد و artifactهای لازم را بررسی و فقط فایل‌های مرتبط را commit کنید. endpoint جدید را در مرجع و مسیر استفاده‌اش را در راهنما ثبت کنید.

تست‌ها چه چیزی را ثابت می‌کنند؟

مجموعه	نمونه فایل واقعی	مورد اثبات
Unit	`OtpChallengeTests`، `ExamAnswerModelTests`، `DetailedReportServiceTests`	قانون خالص، normalization، state و orchestration بدون ادعای رفتار SQL
Integration	`AcademicEndpointsTests`، `EvaluationConcurrencyEndpointsTests`، `ExamAnswerEndpointsTests`	HTTP واقعی، SQL Server، host، constraints، مجوز، نسخه، retry و transaction
Architecture	`OnionDependencyTests`، `LayerPlacementTests`	جهت وابستگی و محل درست لایه‌ها
Hosting	`ProductionHostingTests`	رفتار تنظیمات میزبان در Production در چارچوب تست؛ اثبات استقرار IIS واقعی نیست

`MakanAppWebApplicationFactory` از Windows LocalDB و نام اختصاصی `MakanApp_MessagingSafety_IntegrationTests_Step7E` استفاده می‌کند. `SqlServerCollection` fixture مشترک مجموعه را تعریف می‌کند. initialize دیتابیس تست را پاک و migrationها را اجرا می‌کند؛ dispose نیز همان پایگاه ایزوله را پاک می‌کند. از این نام برای داده شخصی/توسعه استفاده نکنید و همزمان دو اجرای مستقل کل suite را روی همان fixture شروع نکنید.

fixture زمان کنترل‌شده، storage موقت و fault injection برای SMS/file/realtime دارد. از همین ابزارها برای تست زمان/خطای provider استفاده کنید؛ ساعت سیستم را دستکاری یا SMS واقعی در تست ارسال نکنید. تست seedهای داخلی fixture را برای ساخت نقش‌ها و روابط استفاده می‌کند؛ این helperها endpoint محصول نیستند.

فرمان‌های متمرکز زیر فقط برای بازخورد سریع هنگام توسعه‌اند؛ قبل از تحویل، suite کامل قواعد مخزن اجرا شود:

```
dotnet test tests/MakanApp.UnitTests/MakanApp.UnitTests.csproj --configuration Debug --filter
FullyQualifiedName~ExamAnswerModelTests
dotnet test tests/MakanApp.IntegrationTests/MakanApp.IntegrationTests.csproj --configuration Debug --filter
FullyQualifiedName~AcademicEndpointsTests
dotnet test tests/MakanApp.ArchitectureTests/MakanApp.ArchitectureTests.csproj --configuration Debug
```

مسیر عیب‌یابی یک درخواست ناموفق

نشانه	نقطه شروع	اقدام بعدی
host بالا نمی‌آید	Program/DependencyInjection و setting name	خطای validation تنظیمات را بررسی کنید؛ secret را log نکنید
400 با errors	binding و request schema	نوع، فیلد، enum، UUID و format زمان را پیش از service کنترل کنید
401 AUTH_REQUIRED	SessionAuthenticationHandler/IdentityService	expiry، hash key، header و revoke نشست
403 پس از انتخاب role	AccessContext و use case	role، membership، workspace، انتساب کلاس یا رابطه child
404 برای شناسه‌ای که می‌دانید وجود دارد	query scope و سیاست پنهان‌سازی	جست‌وجوی وجود جهانی منبع را جای authorization نگذارید
409	code اختصاصی و state جاری	تضاد lifecycle، ظرفیت، deadline، lease یا operationId متفاوت را تفکیک کنید
412	store و expected rowversion	نسخه جاری را بخوانید؛ داده ورودی کاربر را برای بازبینی حفظ کنید
503 در OTP یا upload	adapter/options	اعتبار سرویس، timeout، مسیر مطلق و permission؛ با500 عمومی یکی نیست
پیام ثبت شده ولی event نرسیده	notifier، outbox و اتصال Hub	receipt/Delta را مرجع قرار دهید و backlog/ خطا را بررسی کنید

نشانه	نقطه شروع	اقدام بعدی
تست SQL شکست می‌خورد	fixture/LocalDB/restore	مشکل محیط را از regression کد جدا گزارش کنید

برای خطای ناشناخته، traceId پاسخ را با log متناظر دنبال کنید. OTP، token، connection string و URL حاوی ServiceToken را در گزارش مشترک قرار ندهید. در EF store مربوط SQL exception، query scope و تغییرات tracked را بررسی کنید؛ نسخه و شناسه مصنوعی مثال‌های این کتاب، داده واقعی پایگاه شما نیستند.

مرز میزبانی و موارد لازم برای تحویل عملیاتی

میزبان migrations را اجرا نمی‌کند؛ health فقط liveness است؛ Swagger/OpenAPI در همه محیط‌ها عمومی‌اند. فایل upload برای دوام بین deploy به RootPath مطلق، permission و storage پایدار نیاز دارد. CORS فعلی wildcard بدون credentials است. handler فقط Bearer header را می‌خواند؛ SignalR WebSocket/SSE مرورگر با query-token از آن پشتیبانی نمی‌گیرد. برای transport قابل آزمایش مرورگر به نمونه LongPolling قرارداد فرانت و راهنمای استقرار مراجعه کنید.

UseForwardedHeaders/trusted proxy صریح در Program ثبت نشده است؛ TLS termination و RemoteIpAddress پشت reverse proxy باید در محیط هدف بررسی شوند. runtime account SQL و principal migration مسئولیت جدا دارند. backup دیتابیس بدون backup فایل‌های referenced کافی نیست. هیچ‌یک از موفقیت تست‌ها یا دستورهای این کتاب به معنی استقرار واقعی، تحویل SMS یا آماده‌بودن چند instance نیست.

تست و عیب‌یابی

اجرای validation

```
dotnet restore MakanApp.sln
dotnet build MakanApp.sln --configuration Debug --nologo
dotnet test MakanApp.sln --configuration Debug --nologo
```

برای تمرکز بر یک مجموعه:

```
dotnet test tests/MakanApp.UnitTests/MakanApp.UnitTests.csproj --configuration Debug --nologo
dotnet test tests/MakanApp.ArchitectureTests/MakanApp.ArchitectureTests.csproj --configuration Debug --nologo
dotnet test tests/MakanApp.IntegrationTests/MakanApp.IntegrationTests.csproj --configuration Debug --nologo
```

Integration از SQL Server LocalDB و پایگاه ثابت `MakanApp_MessagingSafety_IntegrationTests_Step7E` استفاده می‌کند؛ fixture آن را در شروع حذف و با migration بازسازی می‌کند و در پایان حذف می‌کند. این نام فقط برای تست است؛ داده شخصی/توسعه را در آن نگذارید و دو اجرای هم‌زمان این suite را روی یک LocalDB شروع نکنید. test fixture اتصال runtime را برای تست override می‌کند. فایل‌ها هم در پوشه موقت مختص تست نوشته می‌شوند.

Unit قوانین مدل و سرویس‌های قابل تست را بررسی می‌کند. Integration رفتار migration، SQL Server، HTTP، مجوز، همزمانی، idempotency و SignalR را پوشش می‌دهد. Architecture جهت وابستگی و عدم ورود EF Core/ASP.NET Core به لایه‌های داخلی را کنترل می‌کند.

عیب‌یابی سریع

نشانه	بررسی
برنامه با خطای connection string شروع نمی‌شود	تنظیم `ConnectionStrings:MakanDatabase` در محیط واقعی اجرا
`/health` سالم ولی API داده خطا می‌دهد	health فعلی probe دیتابیس نیست؛ اتصال و migration را بررسی کنید
`Invalid object name`	مقصد migration با مقصد runtime یکسان باشد و migrationهای جاری اعمال شده باشند
Swagger در دسترس نیست	در تمام محیطها ثبت می‌شود؛ آدرس، base path و reverse proxy را بررسی کنید
OTP نمی‌رسد	adapter توسعه پیامک واقعی ندارد؛ debugger محلی یا تست خودکار را مطابق بخش ورود به کار ببرید
پس از restart پاسخ `401` می‌گیرید	کلید موقت توسعه عوض شده؛ دوباره وارد شوید یا کلید توسعه را امن و پایدار تنظیم کنید
فضای سازمانی در فهرست نیست	عضویت، سازمان و نقش باید فعال باشند؛ ایجاد حساب جای provisioning را نمی‌گیرد
والد کلاس‌ها را نمی‌بیند	`/academic/classes` برای Parent ارائه نشده؛ برنامه/آزمون/نتیجه از مسیرهای مجاز با context فرزند مصرف شود
نمره بعد از Save دیده نمی‌شود	Draft و Release جدا هستند
پاسخ آزمون ذخیره شده ولی رسید نهایی نیست	Finalize مستقل است و باید پیش از deadline موفق شود
ویدئو یا صدا `415` می‌دهد	`Storage:AllowedContentTypes` پیش‌فرض این MIMEها را ندارد
SignalR مرورگر `401` می‌دهد	محدودیت هدر در مقابل `access_token` Query و transport مورد استفاده
مرورگر خطای CORS دارد	CORS wildcard فعال است؛ credentials:include، HTTP redirect، preflight proxy و header را بررسی کنید؛ موفقیت Postman اثبات اتصال مرورگر نیست
Integration شکست می‌خورد	دسترس‌پذیری LocalDB، اجرای موازی suite و مجوز ایجاد دیتابیس تست

محدودیت‌ها و مسیر مطالعه

مرز قابلیت موجود

در این نسخه provisioning عمومی سازمان/عضویت/دعوت/اشخاص سازمانی/رابطه سرپرست، refresh token، پاک‌سازی زمان‌بندی‌شده فایل‌های auto-finalize، unattached آزمون، پنل backplane، Push provider، moderation چند-instance، Commerce، AI، Copilot، SignalR و export/گزارش زمان‌بندی‌شده API قابل استفاده ندارند. چهار داشبورد، شانزده گزارش تفصیلی، CORS باز، OTP ملی‌پیامک و LocalFileStorage قابل تنظیم در Production موجودند. وجود مدل یا Enum برای یک وضعیت به معنی وجود Endpoint انتقال به آن وضعیت نیست.

تست Integration داده لازم سناریوها را خودش می‌سازد. برای آزمودن دستی کل جریان سازمانی روی دیتابیس تازه، آماده‌سازی داده اولیه یک کار جدا و نیازمند روش مصوب است؛ این راهنما seed یا SQL تولیدی ارائه نمی‌کند.

دیتابیس و منابع کد

Persistence با `MakanDbContext` و schemaهای `identity`، `organization`، `guardian`، `academic`، `assessment`، `storage` و `messaging` سازمان‌دهی شده است. روابط هم‌سازمانی با FKهای ترکیبی، یکتایی وضعیت فعال با filtered unique index و تغییرات حساس با `rowversion` و transaction محافظت می‌شوند. Outbox پیام‌رسانی اعلان را پس از commit کسب‌وکاری ارسال می‌کند.

برای جزئیات مدل، راهنمای ساختار دیتابیس را کنار migrationها بخوانید؛ بازبینی فعلی آن ۵۳ جدول و ۲۰ migration موجود تا انتشار نتیجه آزمون را پوشش می‌دهد. مرجع schema جاری، configurationها و migrationهای موجود است.

مسیر مطالعه یک جریان واقعی آزمون:

1. ExamsController و ExamGradingsController برای قرارداد HTTP.
2. ExamFinalizationService، ExamAnswerService، ExamAttemptService، ExamService و ExamGradingService برای use case.
3. مدل‌های Assessment برای lifecycle و قوانین دامنه.
4. ExamGradingAbstractions برای مرز persistence.
5. پیاده‌سازی Assessment و configurationهای EF.
6. Migrationها برای ساختار SQL Server.
7. ExamGradingEndpointsTests و سایر تست‌های آزمون برای نمونه درخواست و سناریوهای مجاز/غیرمجاز.

برای قرارداد دقیق هر عملیات به مرجع API و برای قواعد کلی مخزن به AGENTS.md مراجعه کنید.

نگهداری همین راهنما و مرجع

Markdownهای `backend-api-guide.md` و `backend-api-reference.md` منابع اصلی‌اند. HTML/PDF هر سند خروجی جداگانه‌اند و برای تحویل کنار منبع نسخه‌بندی می‌شوند. تغییر route، قرارداد، code خطا، مجوز، option، DI یا migration باید در بخش متناظر منعکس شود؛ سپس خروجی‌ها دوباره تولید شوند.

برای تولید خروجی‌ها ابزار مستندسازی مستقل از build محصول است. از ریشه مخزن با Python و مرورگر Edge اجرا کنید؛ وابستگی‌های Python فقط ابزار Markdown/PDF هستند:

```
python -m pip install --target .artifacts/docs-python -r tools/docs/requirements.txt
python -X utf8 tools/docs/build_backend_docs.py --deps .artifacts/docs-python --pdf
```

ابزار Markdown را به HTML مستقل RTL تبدیل می‌کند، هر HTML را با Edge مخفی چاپ می‌کند و PDF دارای شماره صفحه/نشانک می‌سازد. ورودی و خروجی‌ها هر دو در `docs` هستند؛ فایل‌های موقت، گزارش audit و تصاویر کنترل در `.artifacts/backend-handbook` می‌مانند. برای HTML تنها، `--pdf` را حذف کنید. ابزار به SQL Server وصل نمی‌شود و schema یا داده محصول را تغییر نمی‌دهد.

پس از تولید، فهرست، لینک داخلی، فارسی، جدول بلند، نمونه کد و صفحات نماینده PDF را بررسی کنید. اگر سورس محصول تغییر کرده، علاوه بر renderer باید محتوای Markdown با Controller/DTO/سرویس و OpenAPI تازه تطبیق داده شود؛ چاپ دوباره به‌تنهایی سند را به‌روز نمی‌کند.

شناسنامه بازبینی و نتیجه اعتبارسنجی

این تحویل در `2026-10-12` بر مبنای سورس `main@f0ff3be` انجام شده است. تغییرات این مرحله مستندات و ابزار ساخت خروجی‌اند؛ کد محصول، frontend و schema تغییر نکرده‌اند. هیچ migration تازه ساخته یا روی دیتابیس عادی/Production اعمال نشده است.

restore موفق بود؛ build اولیه و نهایی هر دو صفر warning و صفر error داشتند. در هر دو دور ۶۶۵ suite، تست اجرا و موفق شدند: ۲۳۴ Integration، ۴۱۷ Unit و ۱۴ Architecture؛ شکست و skipped صفر. Integration از LocalDB ایزوله fixture استفاده کرد؛ SMS واقعی یا استقرار روی سرور انجام نشد.

۱۴۱ عملیات و پارامترهای query با Controller و OpenAPI تازه میزبان تطبیق داده شدند؛ ۱۸۲ schema JSON و چهار مدل query کنترل شدند. ۱۹۰ نمونه JSON درخواست/پاسخ از نظر فیلد، UUID، nullable، enum، type، nesting، زمان و نسخه بررسی شدند. لینک‌های داخلی/مسیرهای source و تعداد ستون جدول‌ها نیز کنترل شدند. PDF هر کتاب با مرورگر رندر، شماره صفحه و نشانک اضافه و برای صفحه خالی و متن خارج صفحه بررسی شد؛ صفحات نماینده نیز بصری بازبینی شدند.

این بررسی، قرارداد نسخه مشخص مخزن را تأیید می‌کند. آمادگی داده اولیه سازمان، مجوز حساب‌های واقعی، اعتبار ServiceToken، تحویل SMS، storage پایدار و تنظیم proxy باید در محیط هدف جدا آزمایش شوند.